

**Optimización de la gestión de memoria en simuladores cuánticos mediante la
compresión de datos sin pérdida de precisión**

Daniel Adrián González Buendía

Javier Andrés Sarmiento Salazar

Trabajo de grado para optar el título de Ingeniero de Sistemas

Director

Luis Alberto Núñez Martínez

PhD en Ciencias de la Computación

Codirector

Gilberto Javier Díaz Toro

PhD en Ciencias de la Computación

Universidad Industrial de Santander

Facultad De Ingenierías Fisicomecánicas

Escuela De Ingeniería De Sistemas e Informática

Pregrado en Ingeniería de Sistemas e Informática Bucaramanga

2026

Dedicatoria

Este trabajo está dedicado a las personas que acompañaron este proceso de formación personal y profesional.

A nuestras familias.

A nuestras amistades.

A quienes hicieron posible este camino.

Contenido

Introducción	5
1. Planteamiento del problema	6
2. Objetivos	8
2.1. Objetivo general	8
2.2. Objetivos específicos	8
3. Marco teórico	9
3.1. Representación del estado cuántico	9
3.2. Evolución unitaria y simulación tipo Schrödinger	10
3.3. Patrones de acceso inducidos por las compuertas cuánticas	10
3.4. Alternativas para reducir memoria en simulación cuántica clásica	11
3.5. Compresión de datos científicos en punto flotante	12
3.6. Compresión por bloques y jerarquía de memoria	13
3.7. Exactitud numérica y métricas de comparación	14
4. Estado del arte	16
4.1. Simuladores cuánticos	16
4.2. Compresión de memoria con pérdida de precisión	17
4.3. Compresión de memoria sin pérdida de precisión	18
4.4. Algoritmos de compresión sin pérdida de precisión	20
4.4.1. FPC (Burtscher & Ratanaworabhan)	20
4.4.2. FPZIP (Lindstrom & Isenburg)	21
4.4.3. Filtro Bitshuffle con LZ4 (Masui et al.)	22
4.4.4. Transformación de Datos Tipados (TDT)	23
4.5. Otros métodos de optimización de memoria	24

5. Metodología de la investigación	25
5.1. Diseño	26
5.2. Implementación e integración de la estrategia	27
5.3. Evaluación experimental	27
6. Selección del simulador cuántico base	29
6.1. Necesidad de seleccionar un simulador base	29
6.2. Criterios de selección	30
6.3. Simuladores considerados	32
6.4. Matriz de evaluación ponderada	33
6.5. Selección del simulador	34
7. Selección de la biblioteca de compresión sin pérdida	35
7.1. Necesidad de seleccionar una biblioteca de compresión	35
7.2. Criterios de selección	36
7.3. Bibliotecas de compresión sin pérdida consideradas	37
7.4. Matriz de evaluación ponderada	38
7.5. Selección de la biblioteca de compresión	39
8. Patrones de compresibilidad en vectores de estado de algoritmos cuánticos	40
8.1. Relación entre estructura del estado y compresibilidad	40
8.2. Búsqueda de Grover: uniformidad y baja entropía	41
8.3. Transformada cuántica de Fourier: variación de fase y baja redundancia	42
8.4. Algoritmo de Shor: periodicidad y concentración espectral	42
8.5. Deutsch–Jozsa y Simon: estructura y esparsidad	43
8.6. Circuitos variacionales y circuitos aleatorios: entre regularidad y alta entropía	43
8.7. Implicaciones para la compresión sin pérdida	44

9. Diseño de la arquitectura de compresión sin pérdida para simuladores tipo

Schrödinger	44
9.1. Objetivos de diseño	45
9.2. Representación por bloques del vector de estado	45
9.3. Descompresión selectiva y estrategia de acceso	46
9.4. Caché LRU de bloques descomprimidos	47
9.5. Flujo de compresión y descompresión	47
9.6. Ventajas, parámetros y limitaciones del diseño	48

10. Implementación del esquema de almacenamiento comprimido basado en

Blosc	49
10.1. Alcance de la implementación	49
10.2. Realización del esquema de almacenamiento comprimido con Blosc	50
10.3. Disposición en bloques y mapeo del vector de estado	51
10.4. Caché LRU de bloques descomprimidos	51
10.5. Estrategia de acceso y actualización local	52
10.6. Optimización específica para Grover mediante parametrización reducida	53

11. Evaluación experimental y resultados

11.1. Diseño experimental	56
11.2. Resultados para Grover	58
11.2.1. Objetivo único	58
11.2.2. Multiobjetivo	59
11.2.3. Comparación complementaria con la variante optimizada	60
11.3. Resultados para QFT	62
11.3.1. Superposición periódica	62
11.3.2. Superposición de alta entropía	63
11.4. Comparación global de estrategias	64

12.Conclusiones y trabajo futuro	66
12.1. Conclusiones	66
12.2. Trabajo futuro	67
Referencias	72

Lista de figuras

1.	Metodología de investigación	26
2.	Contraste conceptual entre almacenamiento denso y almacenamiento por bloques comprimidos	36
3.	Espacio reservado para visualización comparativa de bibliotecas de compresión	39
4.	Arquitectura abstracta para integración de compresión sin pérdida en simuladores tipo Schrödinger	45
5.	Particionamiento abstracto del vector de estado en bloques independientes .	46
6.	Esquema conceptual de la caché LRU de bloques descomprimidos	47
7.	Flujo abstracto de acceso y actualización de bloques comprimidos	48
8.	Arquitectura del desarrollo implementado para el esquema basado en Blosc .	49
9.	Mapeo entre estados base y bloques físicos en el esquema comprimido	51
10.	Secuencia operativa del esquema basado en Blosc	52

Lista de tablas

1.	Criterios usados para la selección del simulador cuántico	31
2.	Evaluación de alternativas (escala 1–5) y total ponderado	33
3.	Criterios para la selección de bibliotecas de compresión sin pérdida	37
4.	Evaluación de bibliotecas de compresión sin pérdida (escala 1–5) y total ponderado	39
5.	Cargas de trabajo utilizadas en la evaluación	57
6.	Configuraciones de compresión utilizadas en toda la campaña experimental .	57
7.	Grover con objetivo único: promedios sobre las posiciones $N/8$, $N/2$ y $7N/8$	59
8.	Grover multiobjetivo con tres estados marcados	60
9.	Variante optimizada de Grover con objetivo único	61
10.	Variante optimizada de Grover multiobjetivo	61
11.	QFT con superposición periódica	62
12.	QFT con superposición de alta entropía	63
13.	Síntesis comparativa frente al almacenamiento no comprimido	64

Lista de apéndices

A. Derivación de la parametrización reducida de Grover	69
---	-----------

Glosario

Blosc2: Biblioteca de compresión por bloques utilizada en este trabajo para el almacenamiento sin pérdida

HPC: Computación de alto rendimiento (High Performance Computing)

LRU: Least Recently Used, política de reemplazo de caché basada en el uso más reciente

MPI: Message Passing Interface, estándar de comunicación para paralelismo en memoria distribuida

MPS: Matrix Product States, representación tensorial eficiente cuando el entrelazamiento del estado es limitado

OpenMP: API de paralelismo de memoria compartida para programación multihilo

QFT: Transformada Cuántica de Fourier (Quantum Fourier Transform)

ZFP: Biblioteca de compresión para arreglos de punto flotante, empleada aquí como referencia de compresión con pérdida controlada

Resumen

Título: Optimización de la gestión de memoria en simuladores cuánticos mediante la compresión de datos sin pérdida de precisión¹

Autores: Daniel Adrián González Buendía
Javier Andrés Sarmiento Salazar²

Palabras clave: Computación cuántica, Simulación cuántica, Gestión de memoria, Compresión de datos, Computación de Alto Rendimiento (HPC)

Descripción:

La simulación de sistemas cuánticos en hardware clásico enfrenta una limitación severa debido al crecimiento exponencial en la memoria requerida. A medida que aumenta el número de qubits, la cantidad de datos a almacenar supera rápidamente las capacidades de las computadoras convencionales y supercomputadoras. Esta restricción impide la exploración de algoritmos cuánticos avanzados y dificulta la validación de experimentos en entornos clásicos antes de su ejecución en hardware cuántico. Por ello, es fundamental desarrollar estrategias eficientes de gestión de memoria que permitan extender el alcance de estas simulaciones sin comprometer la exactitud numérica de los cálculos. A pesar de los avances en simuladores cuánticos y técnicas de optimización, la gestión de memoria sigue siendo un cuello de botella significativo. En particular, muchas soluciones actuales sacrifican exactitud a cambio de eficiencia, lo que limita su aplicabilidad en contextos donde la precisión es fundamental. Por ello, es necesario explorar estrategias que reduzcan el consumo de memoria sin comprometer la corrección de los resultados. Este trabajo diseñó e implementó una estrategia de compresión de memoria sin pérdida de precisión para simuladores cuánticos tipo Schrödinger. La estrategia se integró en TMFQSfullstate mediante particionamiento por bloques, descompresión selectiva, caché LRU de bloques activos y uso de Blosc2 como biblioteca de compresión. La evaluación experimental, realizada sobre Grover y QFT entre 22 y 24 qubits,

¹Trabajo de grado

²Facultad De Ingenierías Fisicomecánicas. Escuela De Ingeniería De Sistemas e Informática.
Director: PhD. Luis Alberto Núñez Martínez, Codirector: PhD. Gilberto Javier Díaz Toro

mostró que la utilidad de la compresión depende de la estructura del estado: en cargas con alta regularidad se obtuvieron ahorros sustanciales de memoria preservando igualdad exacta frente a la referencia no comprimida, mientras que en estados de alta entropía la compresión dejó de ser conveniente. En conjunto, los resultados establecen que la compresión sin pérdida es una alternativa viable cuando la memoria es la restricción dominante y el estado conserva redundancia explotable.

Abstract

Title: Optimization of memory management in quantum simulators through data compression without loss of precision³

Authors: Daniel Adrián González Buendía
Javier Andrés Sarmiento Salazar⁴

Key words: Quantum Computing, Quantum Simulation, Memory Management, Data Compression, High-Performance Computing (HPC)

Description:

The simulation of quantum systems on classical hardware faces a severe limitation due to the exponential growth in memory requirements. As the number of qubits increases, the amount of data to be stored quickly exceeds the capabilities of conventional computers and supercomputers. This restriction hinders the exploration of advanced quantum algorithms and complicates the validation of experiments in classical environments before their execution on quantum hardware. Therefore, it is crucial to develop efficient memory management strategies that extend the scope of these simulations without compromising numerical exactness. Despite advances in quantum simulators and optimization techniques, memory management remains a significant bottleneck. In particular, many current solutions trade accuracy for efficiency, limiting their applicability in contexts where exact results are crucial. Therefore, it is necessary to explore strategies that reduce memory consumption without compromising the correctness of the simulation. This work designed and implemented a lossless memory-compression strategy for Schrödinger-style quantum simulators. The strategy was integrated into TMFQSfullstate through block partitioning, selective decompression, an LRU cache of active blocks, and Blosc2 as the compression library. The experimental campaign, conducted on Grover and QFT circuits from 22 to 24 qubits, showed that compression usefulness depends on state structure: highly regular workloads achieved substantial memory savings

³Degree Thesis

⁴Faculty of Physics-Mechanics Engineering. School of Systems Engineering and Computer Science.
Advisor: PhD. Luis Alberto Núñez Martínez, Co-Advisor: PhD. Gilberto Javier Díaz Toro

while preserving exact equality against the uncompressed reference, whereas high-entropy states made compression impractical. Overall, the results establish lossless compression as a viable alternative when memory is the dominant constraint and the simulated state preserves exploitable redundancy.

Introducción

La simulación cuántica en computadoras clásicas enfrenta un desafío fundamental debido al crecimiento exponencial del espacio de estados conforme aumenta el número de qubits. La representación explícita de un sistema de 40 qubits en doble precisión requiere aproximadamente 16 TB de memoria RAM, mientras que para 50 qubits la demanda se sitúa ya en el rango de los petabytes, superando la capacidad de las computadoras convencionales (Wu y cols., 2019). Esta limitación restringe el estudio de algoritmos cuánticos avanzados y su validación en hardware clásico antes de su ejecución en procesadores cuánticos (Zhang y cols., 2024). En consecuencia, la gestión eficiente del vector de estado constituye un problema central para ampliar el alcance práctico de la simulación cuántica.

Considerando la creciente importancia de la computación cuántica, optimizar la memoria en simuladores cuánticos no solo permite ampliar el rango de algoritmos simulables en hardware clásico, sino que también reduce la dependencia exclusiva de procesadores cuánticos experimentales. Además, una gestión eficiente de memoria reduce costos computacionales y amplía el número de escenarios que pueden estudiarse en estaciones de trabajo o infraestructuras de alto desempeño.

Para abordar este problema, la literatura ha explorado varias familias de soluciones. Entre ellas se encuentran la compresión con pérdida controlada, mediante bibliotecas como SZ y ZFP, que reduce la memoria a costa de introducir perturbaciones acotadas en las amplitudes (Wu y cols., 2018); los métodos de poda dinámica del estado, que eliminan amplitudes consideradas poco relevantes y modifican la evolución numérica del sistema (Díaz, Steffemel, Barrios, y Couturier, 2025); y los enfoques de computación de alto desempeño, que combinan distribución del vector de estado, paralelismo y optimizaciones de acceso para escalar la simulación sobre infraestructuras de supercomputación (Díaz, Steffemel, Barrios, y Couturier, 2024).

A pesar de estos avances, no todas las estrategias responden al mismo objetivo. Cuando

se requiere conservar exactamente las amplitudes originales, la compresión sin pérdida adquiere interés propio como alternativa sobre la misma formulación de vector de estado. En este trabajo se diseñó e integró una estrategia de almacenamiento comprimido sin pérdida en TMFQSfullstate, basada en particionamiento por bloques, descompresión selectiva, caché LRU e incorporación de Blosc2 en la capa de almacenamiento. La evaluación experimental se realizó sobre Grover y QFT, con un contraste complementario frente a una estrategia con pérdida controlada basada en ZFP, ejecutando las cargas de trabajo sobre el mismo simulador y comparando el estado completo exportado frente a la referencia no comprimida. Los resultados muestran que la conveniencia de comprimir depende de la estructura del estado: en escenarios regulares la estrategia logra ahorros sustanciales de memoria preservando igualdad exacta frente a la referencia no comprimida, mientras que en estados de alta entropía la compresión deja de ser favorable. Sobre esta base, las secciones siguientes desarrollan los fundamentos, el diseño, la implementación y la evaluación de la estrategia propuesta.

1 Planteamiento del problema

La simulación de sistemas cuánticos en computadoras clásicas está condicionada por el crecimiento exponencial de la memoria requerida. Representar un estado cuántico de n qubits implica almacenar 2^n números complejos, lo que rápidamente supera la capacidad de los sistemas de memoria convencionales. En este contexto, técnicas como la poda de estados y la compresión con pérdida reducen el consumo de recursos, pero lo hacen modificando la representación numérica del estado y, por tanto, alterando el régimen de exactitud de la simulación.

La simulación exacta sigue siendo necesaria para validar algoritmos cuánticos, contrastar implementaciones y estudiar su comportamiento antes de ejecutarlos en hardware cuántico. Por ello, el problema no consiste solo en reducir memoria, sino en hacerlo sin introducir discrepancias en las amplitudes del estado cuando la exactitud completa es un requisito. Bajo esta formulación, resulta pertinente estudiar estrategias de compresión sin pérdida que

permitan mejorar el uso de recursos computacionales preservando la igualdad exacta frente a una ejecución de referencia no comprimida.

Con base en este problema, se plantean los objetivos que orientan el desarrollo del trabajo.

2 Objetivos

2.1 Objetivo general

- Diseñar e implementar una estrategia eficiente de gestión de memoria en simuladores cuánticos basada en técnicas de compresión sin pérdida de precisión, con el fin de optimizar el uso de recursos computacionales sin comprometer la exactitud numérica de las simulaciones.

2.2 Objetivos específicos

- Analizar el impacto del uso de herramientas de compresión de datos sin pérdida de precisión en el consumo de memoria, el tiempo de ejecución y la verificación de igualdad exacta de los resultados respecto a una referencia no comprimida.
- Diseñar una estrategia integral de gestión de memoria basada en compresión sin pérdida de precisión, definiendo el algoritmo, los parámetros de compresión-descompresión y los puntos de integración con el simulador cuántico seleccionado.
- Desarrollar e integrar la estrategia diseñada en el simulador elegido, optimizando la sobrecarga computacional para garantizar un ahorro significativo de memoria sin afectar la exactitud numérica de la simulación.
- Implementar dos algoritmos cuánticos distintos en un simulador cuántico para evaluar el uso de memoria, velocidad y exactitud con y sin compresión de datos.
- Comparar los resultados obtenidos con estrategias tradicionales y métodos que emplean compresión con pérdida de precisión, identificando ventajas y desventajas de cada enfoque.

3 Marco teórico

La simulación cuántica clásica de circuitos permite estudiar algoritmos, validar resultados y analizar costos computacionales antes de su ejecución en hardware cuántico. Entre los modelos disponibles, la simulación basada en vectores de estado ocupa un lugar central por su generalidad, aunque dicha generalidad viene acompañada de un crecimiento exponencial en memoria y tiempo. En este contexto, resulta necesario revisar los fundamentos de la representación del estado cuántico, los principales enfoques de simulación clásica, los conceptos de compresión de datos científicos y los criterios de exactitud numérica empleados para evaluar representaciones comprimidas y no comprimidas de un mismo estado.

3.1 Representación del estado cuántico

En el modelo de circuitos cuánticos, el estado de un sistema de n qubits se representa mediante un vector de dimensión 2^n :

$$|\psi\rangle = \sum_{i=0}^{2^n-1} \alpha_i |i\rangle,$$

donde cada amplitud $\alpha_i \in \mathbb{C}$ satisface la condición de normalización

$$\sum_{i=0}^{2^n-1} |\alpha_i|^2 = 1.$$

Esta formulación expresa al sistema como una combinación lineal de estados base del espacio de Hilbert. En términos computacionales, cada amplitud debe almacenarse en memoria clásica cuando se utiliza una representación explícita del vector completo. Si se emplea doble precisión, cada amplitud compleja requiere 16 bytes: 8 para la parte real y 8 para la parte imaginaria. El costo teórico de memoria queda entonces dado por:

$$M(n) = 2^n \times 16 \text{ bytes}.$$

El crecimiento exponencial de $M(n)$ constituye una de las principales limitaciones de la simulación cuántica clásica. Para valores moderados de n , el almacenamiento del vector de estado ya exige cantidades de memoria comparables con las de sistemas de cómputo de alto desempeño. Por esta razón, la representación del estado no solo es un concepto matemático fundamental, sino también el origen del problema computacional tratado en simulación cuántica a gran escala.

3.2 Evolución unitaria y simulación tipo Schrödinger

La evolución de un sistema cuántico aislado se modela mediante operadores unitarios. Si U representa una compuerta cuántica o una secuencia de compuertas, el nuevo estado del sistema se obtiene como

$$|\psi'\rangle = U |\psi\rangle.$$

La simulación tipo Schrödinger consiste en mantener explícitamente el vector de estado y aplicar sobre él las transformaciones unitarias del circuito. Su principal ventaja es la generalidad: cualquier circuito que pueda expresarse en el modelo de compuertas puede, en principio, simularse sin imponer restricciones sobre el grado de entrelazamiento o la forma del estado intermedio. Esta propiedad convierte a la simulación Schrödinger en una base natural para contrastar representaciones alternativas del mismo estado cuántico.

Su principal limitación es que tanto el costo de memoria como una parte importante del costo temporal crecen exponencialmente con el número de qubits. En consecuencia, cualquier estrategia orientada a reducir el uso de memoria debe analizarse sin perder de vista el formalismo de evolución unitaria que define el problema original.

3.3 Patrones de acceso inducidos por las compuertas cuánticas

Aunque el vector de estado contiene 2^n amplitudes, las operaciones elementales de un circuito no siempre requieren tratar todas las entradas de manera arbitraria al mismo tiempo. Una compuerta de un solo qubit actúa sobre pares de amplitudes cuyos índices difieren en

el bit asociado al qubit objetivo. De manera similar, una compuerta de dos qubits actúa sobre grupos de cuatro amplitudes relacionadas por las combinaciones binarias de los qubits involucrados.

Desde la perspectiva algorítmica, esto significa que la actualización del estado presenta patrones estructurados de acceso. La importancia de esta observación reside en el hecho de que el costo práctico de simular un circuito depende no solo del tamaño total del vector, sino también de cómo se distribuyen y reutilizan los accesos sobre sus componentes.

El concepto de localidad resulta entonces relevante. En computación, la localidad temporal describe la tendencia de ciertos datos a reutilizarse en un intervalo corto de tiempo, mientras que la localidad espacial se refiere a accesos cercanos dentro de una misma región de memoria. Ambas nociones son fundamentales para comprender por qué algunas representaciones del estado pueden explotarse con mayor eficiencia que otras.

En simulación tipo Schrödinger, la localidad no implica necesariamente que los elementos afectados se encuentren siempre contiguos en memoria, sino que las compuertas inducen relaciones regulares entre índices. Dichas regularidades permiten razonar sobre patrones repetitivos de lectura y escritura, y por ello resultan importantes cuando más adelante se analizan estrategias de almacenamiento para vectores de gran tamaño.

3.4 Alternativas para reducir memoria en simulación cuántica clásica

La literatura ha propuesto diversos enfoques para reducir el costo espacial de la simulación cuántica. Las redes tensoriales y métodos relacionados aprovechan estructuras favorables del circuito, como bajo entrelazamiento o geometrías particulares, para evitar la materialización explícita del vector completo. De forma análoga, los diagramas de decisión cuánticos explotan redundancias algebraicas y simetrías que permiten representar ciertos estados de manera compacta.

También existen técnicas basadas en partición del circuito, formulaciones tipo caminos de Feynman y esquemas híbridos espacio-tiempo, donde la memoria se reduce al costo de

incrementar el tiempo de cómputo o de restringir la clase de circuitos que pueden tratarse eficientemente. Estas alternativas demuestran que el problema de memoria puede abordarse desde distintos formalismos, pero también evidencian que las ventajas obtenidas suelen depender de propiedades estructurales específicas del circuito o del estado.

Cuando se desea conservar la representación explícita del vector de estado, otra línea de trabajo consiste en estudiar estrategias de almacenamiento más eficientes sobre la misma formulación Schrödinger. En ese caso, el interés teórico deja de estar en sustituir el modelo de simulación y pasa a concentrarse en cómo representar, transformar y recuperar los datos numéricos del estado sin alterar su significado físico.

Esta distinción es importante porque no todas las técnicas de reducción de memoria persiguen el mismo objetivo. Algunas cambian la representación matemática del estado para aprovechar estructura; otras mantienen la misma representación y modifican únicamente la manera en que los datos se guardan, se recuperan o se transfieren en memoria. Diferenciar ambos enfoques ayuda a interpretar con mayor claridad las decisiones presentadas en capítulos posteriores.

3.5 Compresión de datos científicos en punto flotante

La compresión de datos busca reducir el espacio requerido para almacenar o transmitir información. En términos generales, puede clasificarse en dos grandes categorías: compresión sin pérdida y compresión con pérdida. La primera garantiza la reconstrucción exacta de los datos originales tras la descompresión; la segunda acepta una perturbación controlada a cambio de mayores ratios de compresión.

En datos científicos representados en punto flotante, la distinción entre ambos enfoques es especialmente importante. Muchas aplicaciones numéricas toleran errores acotados y, por ello, emplean compresión con pérdida. Sin embargo, cuando se requiere preservar exactamente los valores originales, solo la compresión sin pérdida resulta admisible.

Los arreglos de punto flotante presentan desafíos particulares para la compresión porque

con frecuencia exhiben alta entropía aparente. No obstante, incluso en secuencias numéricas complejas pueden existir regularidades en su representación binaria, por ejemplo en signos, exponentes o patrones a nivel de bytes. Por esta razón, una práctica común en compresión científica consiste en aplicar primero transformaciones reversibles que reorganicen los datos y, posteriormente, utilizar compresores sin pérdida de propósito general o de alta velocidad.

Técnicas como *shuffle* y *bitshuffle* son ejemplos de este principio. Ambas reorganizan los bytes o los planos de bits de un conjunto de valores para hacer más visibles ciertas regularidades al compresor posterior. Este tipo de preconditionamiento es particularmente relevante en arreglos homogéneos de gran tamaño, como los que aparecen en simulación numérica y cómputo científico.

Desde el punto de vista conceptual, estas transformaciones no cambian el significado de los datos, sino únicamente su organización antes de comprimir. Su utilidad radica en que muchos compresores funcionan mejor cuando reciben secuencias con patrones repetitivos explícitos. En consecuencia, una parte importante del problema no depende solo del compresor elegido, sino también de cuán favorable resulta la disposición binaria de los datos para dicho compresor.

3.6 Compresión por bloques y jerarquía de memoria

En el manejo de grandes volúmenes de datos científicos, una estrategia frecuente consiste en particionar los arreglos en bloques independientes. Este principio aparece en técnicas de almacenamiento, procesamiento fuera de núcleo, cachés y bibliotecas de compresión orientadas a alto desempeño. La motivación general es que operar sobre unidades más pequeñas permite equilibrar mejor el costo de acceso, el aprovechamiento de la localidad y el uso de memoria temporal.

La granularidad del bloque introduce un compromiso clásico. Bloques grandes tienden a capturar más redundancia y pueden favorecer mejores ratios de compresión, pero suelen incrementar la latencia de acceso cuando solo una fracción pequeña de los datos es nece-

saria. Bloques pequeños reducen el costo de acceso localizado, aunque pueden degradar el rendimiento del compresor y aumentar el peso relativo del metadato y la administración.

Asociado a esta idea aparece el concepto de jerarquía de memoria. En muchos sistemas de cómputo, una parte de los datos permanece en almacenamiento principal, mientras otra se mantiene temporalmente en niveles de acceso más rápido. Las decisiones sobre reemplazo, reutilización y permanencia de los datos en estos niveles intermedios se relacionan con la noción general de caché. Dentro de este marco, políticas como *Least Recently Used* (LRU) constituyen mecanismos clásicos para explotar localidad temporal cuando el conjunto de datos activo excede la capacidad de memoria inmediata.

La intuición detrás de LRU es sencilla: si un dato fue utilizado recientemente, es razonable suponer que tiene mayor probabilidad de reutilizarse en el corto plazo que otro que lleva más tiempo sin accederse. Por ello, cuando la capacidad disponible es limitada, la política expulsa primero el elemento menos recientemente utilizado. Aunque esta regla no siempre produce la decisión óptima, su fundamento coincide con un principio ampliamente aceptado en sistemas de memoria: muchos programas exhiben conjuntos de trabajo que permanecen activos durante intervalos acotados de ejecución.

En este sentido, el valor teórico de una política como LRU no depende de una implementación particular, sino de su relación con la noción de localidad temporal. Comprender este vínculo facilita interpretar por qué la reutilización reciente de datos puede traducirse en menores costos de acceso y, en términos generales, en una mejor utilización de memoria intermedia.

3.7 Exactitud numérica y métricas de comparación

La evaluación de representaciones comprimidas y no comprimidas de un mismo estado requiere distinguir entre ahorro de memoria, costo temporal y exactitud numérica. En cuanto

a memoria, una métrica habitual es la razón de compresión:

$$CR = \frac{S_{\text{sin comp.}}}{S_{\text{comp.}}},$$

donde $S_{\text{sin comp.}}$ representa el tamaño del estado en su forma no comprimida y $S_{\text{comp.}}$ el tamaño total bajo compresión.

En cuanto a desempeño, resulta natural medir el tiempo total de ejecución y la sobrecarga relativa frente a una simulación equivalente sin compresión:

$$OH = \frac{T_{\text{comp}} - T_{\text{sin comp.}}}{T_{\text{sin comp.}}}.$$

Cuando el objetivo es preservar exactamente las amplitudes originales, la corrección debe formularse en primer lugar como una condición de igualdad exacta respecto a una simulación de referencia sin compresión. En términos ideales, esto significa verificar que

$$\alpha_i^{(\text{ref})} = \alpha_i^{(\text{comp})} \quad \forall i.$$

Esta condición expresa de forma directa el significado de una estrategia sin pérdida: el estado recuperado debe coincidir exactamente con el estado de referencia, no solo aproximarse a él. Cuando la simulación utiliza la misma precisión numérica y el mismo orden de operaciones, esta igualdad puede incluso comprobarse a nivel bit a bit.

Como medida auxiliar de reporte, puede emplearse el error absoluto máximo:

$$e_{\text{abs}} = \max_i \left| \alpha_i^{(\text{ref})} - \alpha_i^{(\text{comp})} \right|.$$

Esta métrica no sustituye el criterio de igualdad exacta, pero resulta útil para resumir numéricamente cualquier discrepancia en un único valor. En un esquema estrictamente sin pérdida se espera que $e_{\text{abs}} = 0$.

4 Estado del arte

4.1 Simuladores cuánticos

La simulación clásica de circuitos cuánticos es esencial para el desarrollo y validación de algoritmos cuánticos. Sin embargo, el principal desafío radica en la representación del estado cuántico, cuyo tamaño crece exponencialmente con el número de qubits.

Uno de los primeros enfoques para abordar este problema fue la representación mediante *Matrix Product States* (MPS), propuesta por Vidal (2003). Este método permite representar ciertos estados cuánticos de forma eficiente cuando el entrelazamiento es limitado, reduciendo los requisitos de almacenamiento y cómputo. Su principal ventaja es la escalabilidad en circuitos con baja profundidad, permitiendo simular cientos de qubits. No obstante, su limitación radica en que para sistemas altamente entrelazados el costo computacional crece exponencialmente, restringiendo su aplicabilidad a ciertos algoritmos.

A medida que se buscaban métodos más generales, surgió la simulación tipo *Schrödinger*, que almacena el vector de estado completo y lo actualiza en cada operación cuántica. Este enfoque, empleado en simuladores como QuEST y Qiskit Aer (Jones, Brown, Bush, y Benjamin, 2019; Qiskit Development Team, 2025), garantiza alta precisión y permite simular cualquier circuito cuántico. Su desventaja principal es la extrema demanda de memoria, la cual crece exponencialmente con el número de qubits: una simulación de 45 qubits ya ha requerido 0.5 petabytes de memoria (Häner y Steiger, 2017), y al extender esta representación de doble precisión a 50 qubits el costo pasa al orden de decenas de petabytes.

Para mitigar el problema de memoria, se desarrollaron métodos basados en partición del circuito y caminos de Feynman. Por ejemplo, Chen y cols. presentaron un algoritmo distribuido capaz de calcular amplitudes de salida y simular circuitos universales de hasta 12×12 qubits con profundidad moderada sin materializar el estado completo (Chen, Zhang, Huang, Newman, y Shi, 2018). Sin embargo, aunque reducen el almacenamiento requerido, estos métodos incrementan la complejidad computacional, ya que el número de caminos crece

exponencialmente con la profundidad del circuito.

4.2 Compresión de memoria con pérdida de precisión

Dado que la representación exacta de estados cuánticos es ineficiente en términos de memoria, se han explorado técnicas de compresión con pérdida de precisión. Estos enfoques sacrifican una pequeña cantidad de fidelidad numérica a cambio de una reducción significativa en el uso de memoria, permitiendo la simulación de circuitos más grandes.

Uno de los primeros enfoques en este ámbito fue el de Wu y cols. (2018, 2019), quienes introdujeron la compresión adaptativa con error acotado. Utilizando la biblioteca SZ, lograron reducir el tamaño del vector de estado hasta en un factor de 16, manteniendo fidelidades superiores al 99.9 %. Su principal ventaja es que permite ampliar el número de qubits simulados sin un incremento proporcional en los requisitos de memoria. Sin embargo, la compresión y descompresión recurrente introduce sobrecarga computacional, afectando la eficiencia temporal de la simulación.

En un esfuerzo por maximizar la escala de la simulación cuántica de estado completo, Zhang y cols. (2024) presentaron *BMQSim*, un marco que extiende *SV-Sim* mediante el uso de **compresores con pérdida de precisión y control de error punto a punto** ejecutados directamente en GPU (por ejemplo, variantes de la familia SZ/cuSZ)(Zhang y cols., 2024). Su diseño combina:

- Una estrategia de *circuit partitioning* que disminuye la frecuencia de (des)compresión.
- Un *pipeline* solapado que oculta en gran medida el coste de mover y comprimir datos.
- Una gestión de memoria jerárquica (VRAM + SSD mediante GPUDirect Storage) que asigna dinámicamente espacio a los bloques comprimidos.

Con estas técnicas, *BMQSim* reduce el consumo de memoria en más de un orden de magnitud y mantiene fidelidades superiores al 99 % sin penalizar de forma apreciable el tiempo de simulación. Su eficacia, sin embargo, está condicionada a la disponibilidad de hardware

acelerado con GPUs de alto rendimiento y unidades NVMe rápidas, lo que puede restringir su adopción en plataformas de menor capacidad computacional.

Más recientemente, Huffman, Pavlichin, y Weissman (2024) exploraron la cuantización de amplitudes como un mecanismo para reducir la precisión numérica sin afectar la fidelidad global del sistema. Demostraron que representaciones con tan solo 7 bits por amplitud permiten mantener una fidelidad superior al 99 %, lo que sugiere que la precisión completa de punto flotante puede ser innecesaria en muchas simulaciones. Aunque esta técnica ofrece un enfoque simple y eficiente para reducir el uso de memoria, su aplicabilidad a circuitos de gran escala aún requiere validación empírica.

Díaz Toro (2025) propone un esquema de **vector de estado comprimido** que emplea compresores con pérdida de precisión—principalmente *ZFP* en modo *fixed-rate*, complementado por *SZ*—para reducir la huella de memoria del simulador. El autor reporta reducciones del **10–20 %** en casos base y, en configuraciones de mayor escala, ahorros que alcanzan hasta un **75 %** de la memoria requerida. *ZFP* opera sobre bloques independientes de 4^d valores, de modo que la (des)compresión de cada bloque se realiza en completo aislamiento del resto; esto permite solapar dichas operaciones con el cómputo principal y amortiguar la sobrecarga introducida. Aunque la compresión incrementa el tiempo de ejecución, el impacto se compensa al transmitir los fragmentos del vector en formato comprimido dentro de una arquitectura distribuida basada en *MPI*, reduciendo el volumen de comunicación y habilitando simulaciones de hasta 30 qubits en los experimentos reportados. Finalmente, el estudio concluye que esta estrategia de *estado completo comprimido distribuido* resulta más fiable y escalable que la *poda de estados de baja probabilidad*, la cual presenta descensos de fidelidad y sobrecarga adicional que la hacen desaconsejable.

4.3 Compresión de memoria sin pérdida de precisión

La simulación cuántica eficiente requiere reducir el consumo de memoria sin comprometer la fidelidad del estado cuántico. Para ello, se han desarrollado técnicas de compresión sin

pérdida de precisión que buscan representar estados y operadores cuánticos de manera más compacta sin alterar sus valores.

Uno de los primeros enfoques en esta dirección fue el uso de diagramas de decisión cuánticos (*Quantum Information Decision Diagrams*, QuIDD), introducidos por Viamontes, Markov, y Hayes (2003). Este método permite representar vectores de estado y operadores unitarios aprovechando redundancias estructurales en sus coeficientes, lo que permite una representación más eficiente en memoria. Su ventaja radica en que, para circuitos con alta redundancia, el crecimiento en memoria puede reducirse de exponencial a polinomial. Sin embargo, su eficacia se limita a circuitos con estructura repetitiva, fallando en casos de entrelazamiento complejo.

En un desarrollo posterior, Miller y Thornton (2006) propusieron los *Quantum Multiple-valued Decision Diagrams* (QMDD), los cuales extienden la representación de QuIDD mediante el uso de aristas ponderadas con matrices unitarias. Esto permitió manejar de manera más eficiente circuitos reversibles y operadores cuánticos de mayor tamaño. Aunque estos diagramas optimizan la representación de puertas lógicas y matrices densas, su eficiencia sigue dependiendo de la estructura interna del circuito.

Más recientemente, Jaques y Häner (2021) exploraron la esparsidad del estado cuántico para almacenar solo las amplitudes no nulas, reduciendo significativamente el almacenamiento requerido en algoritmos con exploración limitada del espacio de Hilbert. Su enfoque demostró ser altamente eficiente en problemas de factorización y aritmética cuántica, pero pierde efectividad en circuitos con alta interferencia y entrelazamiento.

En 2023, Vinkhuijzen, Coopmans, Elkouss, Dunjko, y Laarman (2023) introdujeron los *Local Invertible Map Decision Diagrams* (LIMDD), una evolución de los diagramas de decisión que incorpora transformaciones locales para mejorar la representación compacta del estado cuántico. Esta técnica permite manejar estados estabilizadores y ciertas estructuras altamente entrelazadas que eran difíciles de representar con diagramas de decisión tradicionales. No obstante, su aplicabilidad sigue limitada a casos donde las transformaciones locales

puedan ser explotadas para reducir la complejidad estructural.

Estos avances en compresión sin pérdida han permitido extender la capacidad de simulación de circuitos cuánticos en hardware clásico. Sin embargo, aún existen limitaciones en la representación de estados arbitrarios, lo que ha llevado a la exploración de otros métodos para optimizar el uso de memoria.

4.4 Algoritmos de compresión sin pérdida de precisión

En simulaciones científicas y cuánticas es común manejar vectores de estado representados como grandes arreglos de números de punto flotante. Cuando estos datos carecen de correlación temporal o patrones estructurales (es decir, presentan alta entropía o *aleatoriedad*), la compresión sin pérdida se vuelve especialmente desafiante. Muchos algoritmos de compresión de punto flotante aprovechan transiciones suaves o redundancias entre valores consecutivos; por ejemplo, técnicas de serie temporal almacenan solo los bits que difieren entre un valor y el siguiente, explotando que a menudo los valores sucesivos comparten prefijos de bits iguales. Sin embargo, en datos altamente aleatorios, estas suposiciones se rompen y tales métodos tienden a no lograr compresión significativa. Por ello, conviene distinguir entre el nivel *algorítmico*, donde se ubican predictores, transformaciones y códecs, y el nivel de *realización software*, donde dichos mecanismos aparecen empaquetados en bibliotecas utilizables desde un simulador en C/C++. A continuación, se describen los principales algoritmos concebidos para este tipo de datos; la sección de selección posterior traduce este panorama conceptual a alternativas concretas de integración.

4.4.1 *FPC (Burtscher & Ratanaworabhan)*

El algoritmo *FPC* (*Floating Point Compressor*) (Burtscher y Ratanaworabhan, 2009) fue diseñado para la compresión sin pérdida de flujos lineales de datos en doble precisión (64 bits). Internamente, FPC emplea un esquema de *predicción* dual junto con codificación de ceros líderes. Concretamente, mantiene dos predictores (derivados de técnicas FCM/DFCM)

almacenados en tablas hash, que generan un valor estimado para el siguiente número en la secuencia. Para cada nuevo valor de entrada, FPC selecciona la predicción que más coincide en sus bits más significativos con el valor real, y calcula el XOR entre el valor real y el predicho. Este XOR resalta únicamente los bits diferentes, convirtiendo los bits coincidentes en *cero*. A continuación, el algoritmo cuenta cuántos bytes iniciales del resultado XOR son cero (ceros líderes) y codifica esa cantidad en 3 bits, junto con 1 bit adicional que indica cuál predictor fue usado. Estos 4 bits de código, seguidos de los *bytes residuales* no nulos (almacenados sin más codificación), forman la salida comprimida.

Esta estrategia implica que, si el valor predicho es cercano al real, la diferencia tendrá muchos ceros líderes y, por tanto, pocos bytes residuales que almacenar. En escenarios de datos altamente aleatorios (donde las predicciones rara vez aciertan), FPC tiende a obtener pocos ceros líderes y la tasa de compresión se aproxima a 1:1 (sin reducción notable). No obstante, su diseño minimiza la penalización en estos casos: los bloques comprimidos incluyen encabezados mínimos y códigos de apenas 4 bits por valor, garantizando sobrecarga reducida y un rendimiento consistente. Así, aunque FPC no logre compresión sustancial en vectores de estado cuántico aleatorios, ofrece una solución de alta velocidad con mínima sobrecarga, capaz de detectar rápidamente si no hay redundancia aprovechable y manejar dichos datos sin degradar el rendimiento.

4.4.2 *FPZIP (Lindstrom & Isenbug)*

FPZIP (Lindstrom y Isenbug, 2006) es un compresor de propósito específico para arreglos de punto flotante. Su enfoque se basa en la *predicción adaptativa* y la *codificación entrópica* eficiente para lograr compresión sin pérdida en múltiples contextos (mallas no estructuradas, volúmenes 3D, imágenes, etc.). Internamente, FPZIP aplica esquemas de predicción multi-dimensional (por ejemplo, predictores que aprovechan la continuidad espacial de los datos científicos) y luego comprime únicamente el error de predicción. A diferencia de métodos que cuantizan los flotantes a enteros, FPZIP opera directamente sobre las representaciones

IEEE 754, manteniendo exactitud bit a bit.

Su arquitectura soporta predictores ajustables al tipo de datos: por ejemplo, en un campo volumétrico 3D puede usar un predictor basado en valores vecinos, mientras que en una serie temporal podría usar extrapolación simple. Tras la predicción, FPZIP emplea un modelo de codificación entrópica optimizado para velocidad, que comprime los residuos de manera eficaz. Esta combinación le permite alcanzar tasas de compresión y velocidades competitivas, acelerando la E/S de simulaciones científicas al reducir el volumen de datos sin sacrificar precisión. En contextos de datos altamente aleatorios, donde las predicciones en su mayoría fallan, FPZIP tiende a comportarse como un compresor genérico pero con ventajas sutiles: incluso en ausencia de correlación temporal, puede explotar patrones locales dentro de la representación en coma flotante (p. ej., regularidades en campos de exponentes o signos comunes) que los algoritmos genéricos pasan por alto al tratar los datos solo como bytes crudos. Para vectores de estado cuántico, si bien las amplitudes pueden parecer aleatorias, FPZIP podría aprovechar distribuciones no uniformes de ciertos bits, por ejemplo exponentes repetidos, logrando compresiones modestas con un rendimiento robusto.

4.4.3 Filtro Bitshuffle con LZ4 (Masui et al.)

Una estrategia distinta para afrontar datos de punto flotante sin patrones evidentes es emplear transformaciones previas que revelen posibles regularidades a nivel de bits. *Bitshuffle* (Masui y cols., 2015) es un algoritmo de preprocesamiento sin pérdida que extiende el filtro *byte-shuffle* al nivel de *bit*. En lugar de reordenar solo los bytes de cada número, Bitshuffle realiza una *transposición bit a bit* de una matriz de valores: considera la representación binaria de N números de m bits (por ejemplo, $m=32$ para flotantes en simple precisión) como una matriz de $N \times m$, y reorganiza los datos tomando columnas de bits completas. Tras esta permuta, aplica una compresión LZ, usualmente LZ4, sobre la secuencia resultante.

El fundamento es que, incluso si los valores originales parecen aleatorios, puede haber *correlaciones ocultas* dentro de ciertos planos de bits. Por ejemplo, en datos científicos muchos

valores comparten el mismo exponente o presentan bits altos similares, lo que tras Bitshuffle se manifiesta como secuencias repetitivas (*runs* de ceros o unos) más fáciles de comprimir mediante LZ4. A diferencia de métodos predictivos, Bitshuffle no asume continuidad entre elementos adyacentes; su efectividad radica en convertir cualquier correlación intra-byte (dentro de la representación binaria de cada valor) en redundancias horizontales a lo largo de la secuencia transformada. En datos de alta entropía pura (bits totalmente aleatorios), Bitshuffle no puede crear patrones donde no los hay; con todo, en vectores de estado cuántico puede explotar pequeñas irregularidades: si los qubits están normalizados, muchos amplitudes comparten signos o exponentes comunes, generando columnas de bits constantes que LZ4 comprime muy bien. En la práctica, Bitshuffle+LZ4 logra relaciones de compresión superiores a compresores genéricos en escenarios científicos sin reducción de precisión, a la vez que mantiene velocidades de descompresión del orden de gigabytes por segundo por núcleo.

4.4.4 Transformación de Datos Tipados (TDT)

Una innovación reciente en compresión de flotantes es la *Transformación de Datos Tipados* (TDT) (Jamalidinan y Cheshmi, 2025), concebida para cerrar la brecha entre compresores genéricos y especializados en punto flotante. TDT no es un compresor en sí, sino una etapa de *preprocesamiento* adaptable que reorganiza los bytes de los datos antes de aplicar un compresor convencional (p. ej., Zstd o LZ4). La idea clave es agrupar y separar los bytes según su *nivel de entropía* y patrones estadísticos, en lugar de tratarlos uniformemente. En una representación IEEE 754, distintos campos (signo, exponente, mantisa) pueden poseer distribuciones diferentes; por ejemplo, en muchas series científicas los bytes de exponente tienden a repetirse más que los bytes de mantisa. TDT analiza el conjunto de datos (en bloques) extrayendo características estadísticas de cada posición de byte (entropía media, desviación, frecuencias de valores) y aplica *clustering* para decidir cuáles bytes se comportan de forma similar. Luego *reordena* la matriz de datos colocando juntos los bytes de comportamientos afines y separando los de entropía distinta. Esto produce flujos de datos más homogéneos

que, al ser alimentados a un compresor genérico, permiten una codificación más eficiente.

TDT no altera ningún bit de la información; se trata de una transformación reversible cuya utilidad consiste en exponer patrones que antes estaban enmascarados por la disposición original de los datos. En el contexto de vectores de estado cuántico, donde la aleatoriedad aparente puede ser alta, esta técnica buscaría si ciertos bytes de mantisa comparten distribuciones o si el byte de exponente presenta menor entropía debido a normalizaciones. Incluso cuando los datos son mayormente aleatorios, el método puede detectar no uniformidades sutiles entre componentes y reagruparlas para facilitar la compresión. Por ello, resulta una alternativa interesante como capa previa: en conjuntos de baja entropía conserva ventajas de los modelos para punto flotante y, en conjuntos de alta entropía, actúa como un reordenamiento que puede mejorar de forma moderada el rendimiento de compresión y descompresión.

4.5 Otros métodos de optimización de memoria

Además de la compresión sin pérdida, se han desarrollado enfoques alternativos para reducir el consumo de memoria en simuladores cuánticos, incluyendo técnicas basadas en redes tensoriales, poda de estados y optimización HPC.

Uno de los primeros enfoques en esta dirección fue propuesto por Markov y Shi (2008), quienes introdujeron el uso de redes tensoriales para simular circuitos cuánticos mediante contracción optimizada de tensores. En este método, el circuito se representa como un grafo tensorial, y su simulación se optimiza contrayendo secuencias de tensores en un orden eficiente. Esto permite representar ciertos circuitos con memoria subexponencial, aunque la efectividad del método depende de la estructura del entrelazamiento.

Posteriormente, Boixo, Isakov, Smelyanskiy, y Neven (2017) propusieron un modelo gráfico para circuitos cuánticos de baja profundidad basado en la eliminación de variables en grafos probabilísticos. Su técnica permitió simular circuitos aleatorios cercanos al régimen de supremacía cuántica en hardware clásico, extendiendo el tamaño de circuitos simulables sin necesidad de almacenar el estado cuántico completo. Sin embargo, este método pierde

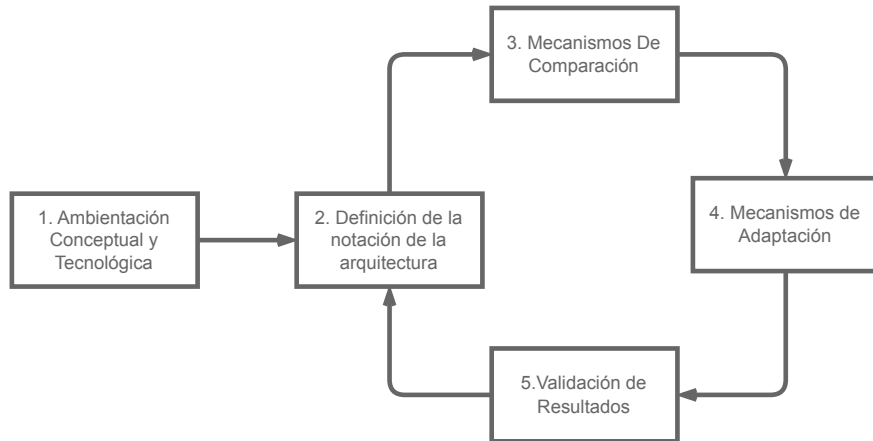
eficacia conforme aumenta la profundidad del circuito.

En paralelo, Pednault y cols. (2017) introdujeron técnicas de diferimiento de contracciones tensoriales en simulaciones HPC. Su propuesta permite manejar circuitos más grandes al intercambiar memoria por cómputo adicional, almacenando resultados intermedios en memoria secundaria y evitando la expansión total del estado cuántico en RAM. Esta estrategia ha sido clave en la simulación de circuitos con estructuras altamente no triviales, aunque su implementación requiere acceso a infraestructuras de supercomputación.

Finalmente, como se analizó previamente, la tesis doctoral de Díaz Toro (2025) introduce técnicas de compresión con pérdida de precisión en esquemas distribuidos de simulación cuántica. Este antecedente resulta especialmente relevante porque consolida el uso de bibliotecas científicas de compresión, vectores de estado y paralelismo distribuido con *MPI* como una línea válida para reducir consumo de memoria en simulación cuántica. Al mismo tiempo, deja visible un espacio de estudio complementario: evaluar hasta qué punto técnicas de compresión **sin pérdida de precisión** pueden aportar reducciones útiles de memoria sin introducir discrepancias numéricas en el estado simulado.

5 Metodología de la investigación

Este estudio se desarrolló mediante una metodología de investigación aplicada con evaluación experimental. El problema abordado exigió diseñar, implementar y validar una estrategia de compresión sin pérdida de precisión sobre un simulador cuántico tipo Schrödinger, manteniendo la exactitud numérica del estado y cuantificando el efecto de la compresión sobre memoria y tiempo. En consecuencia, la metodología combinó revisión del estado del arte, selección justificada de herramientas, diseño de la arquitectura de almacenamiento, integración en el simulador base y experimentación controlada sobre cargas de trabajo representativas.

Figura 1: Metodología de investigación

5.1 Diseño

Comprendió el estudio del estado del arte, la definición de requisitos y supuestos, la selección del simulador y la planificación experimental y de integración técnica.

- **Revisión de literatura y estado del arte.** Se realizó una revisión estructurada de técnicas de compresión sin pérdida de precisión para arreglos de punto flotante y de su aplicabilidad en simulación cuántica de *vectores de estado*. A partir de esa revisión se identificaron ventajas, limitaciones y brechas relevantes para el problema.
- **Criterios y procedimiento de selección del simulador cuántico.** Se evaluaron alternativas como QuEST, Intel-QS, qsim, Quantum++ y TMFQSfullstate con base en apertura y modificabilidad del código, facilidad de integración de estructuras personalizadas para el vector de estado, soporte a optimizaciones de memoria y paralelismo, y capacidad de instrumentación de tiempo y memoria. La decisión se formalizó mediante una matriz ponderada.
- **Arquitectura de la solución y diseño de la estrategia de compresión.** Se definieron las estructuras de datos y los puntos de inserción en la capa de almacenamiento del vector de amplitudes, junto con las políticas de compresión, descompresión y reutilización temporal de bloques requeridas por la integración.

5.2 Implementación e integración de la estrategia

La estrategia de compresión seleccionada se implementó en el simulador cuántico elegido, modificando su sistema de almacenamiento y gestión de memoria. Durante esta etapa se optimizó el esquema de acceso para evitar sobrecostos computacionales innecesarios asociados a la compresión y descompresión. Las actividades principales fueron las siguientes:

- Modificación del código fuente del simulador para integrar la compresión sin pérdida.
- Optimización del sistema de almacenamiento para reducir el uso de memoria y controlar la sobrecarga temporal.
- Validación inicial de la implementación mediante circuitos de prueba y comprobación de igualdad exacta frente a la referencia no comprimida.

5.3 Evaluación experimental

Se diseñó una campaña experimental para evaluar el impacto de la estrategia de compresión sobre circuitos cuánticos con estructuras de estado contrastantes. La evaluación permitió determinar en qué condiciones la compresión sin pérdida ofrece un balance favorable entre ahorro de memoria y costo temporal. En esta fase se realizaron las siguientes actividades:

- Definición de un conjunto de circuitos de prueba para evaluar la estrategia de compresión:
 - Transformada de Fourier Cuántica (QFT)
 - Algoritmo de búsqueda de Grover
- Ejecución de simulaciones con y sin compresión para medir su impacto en el rendimiento y el uso de memoria:
 - Tiempo de ejecución total (segundos)
 - Uso máximo de memoria (MB)

- Verificación de igualdad exacta del estado respecto a la referencia sin compresión:

$$\alpha_i^{(\text{ref})} = \alpha_i^{(\text{comp})} \quad \forall i.$$

- Error absoluto máximo como medida auxiliar de discrepancia:

$$e_{\text{abs}} = \max_i \left| \alpha_i^{(\text{ref})} - \alpha_i^{(\text{comp})} \right|.$$

- Análisis de los resultados obtenidos y contraste con el almacenamiento no comprimido y con una estrategia complementaria de compresión con pérdida controlada.

La campaña experimental se ejecutó sobre la estación de trabajo donde se desarrolló el simulador, equipada con Rocky Linux 10.1, procesador AMD Ryzen 7 5700X de 8 núcleos y 16 hilos, y 32 GiB de memoria RAM. Las métricas de tiempo y memoria residente máxima se recolectaron con **psrecord** a partir de las ejecuciones de los binarios experimentales del simulador. En Grover se promediaron tres instancias por tamaño para reducir la dependencia de la posición de los estados marcados; en los registros empleados, la dispersión temporal entre esas tres posiciones se mantuvo acotada y la memoria pico permaneció prácticamente invariante para una misma estrategia y un mismo tamaño. En QFT se reportó una instancia por familia de entrada y por número de qubits.

La verificación de exactitud para la estrategia sin pérdida se realizó exportando el vector completo de amplitudes de la ejecución de referencia no comprimida y el vector completo de la ejecución con almacenamiento Blosc sobre la misma carga de trabajo, para luego compararlos amplitud por amplitud. En la estrategia con pérdida basada en ZFP, el error reportado se obtuvo mediante el mismo procedimiento de exportación completa, resumiendo la discrepancia con el máximo error absoluto frente a la referencia no comprimida.

6 Selección del simulador cuántico base

6.1 Necesidad de seleccionar un simulador base

El núcleo experimental de este trabajo consiste en diseñar e integrar técnicas de compresión *sin pérdida de precisión* sobre la estructura de datos más costosa en simulación cuántica tipo Schrödinger: el vector de estado (amplitudes complejas). Por lo tanto, el simulador seleccionado no es un detalle de implementación, sino una decisión metodológica que condiciona:

- la **viabilidad técnica** de insertar estructuras de almacenamiento comprimidas sin alterar la fidelidad numérica;
- el **costo de ingeniería** requerido para instrumentar mediciones reproducibles de memoria y tiempo;
- y la **capacidad de aislar variables** (compresión vs. paralelismo vs. optimizaciones internas) durante la evaluación experimental.

De hecho, dentro de la fase *Diseño* planteada en la metodología, se establece explícitamente que la selección debe basarse en: (i) apertura y modificabilidad del código, (ii) facilidad de integrar arreglos/estructuras personalizadas para el vector de estado, (iii) soporte a optimizaciones de memoria y paralelismo, y (iv) capacidad de instrumentación (medición de memoria y tiempo).

Bajo ese marco, se estudian cinco alternativas representativas por uso y disponibilidad en la comunidad: QuEST, Intel-QS, qsim, Quantum++ y TMFQSfullstate. Para asegurar trazabilidad, la decisión se formaliza con criterios y una matriz de decisión ponderada, de modo que la elección final sea justificable y reproducible.

6.2 Criterios de selección

Con el fin de elegir un simulador adecuado para integrar y evaluar compresión *sin pérdida de precisión* sobre el vector de estado, se definieron los criterios de la tabla 1. Estos criterios operacionalizan los lineamientos metodológicos del diseño (modificabilidad, integración de estructuras, optimización/paralelismo e instrumentación), y además incorporan requisitos prácticos de ingeniería para reducir riesgo técnico durante la implementación.

Tabla 1: Criterios usados para la selección del simulador cuántico

	Criterio	Explicación	Peso
C1	Vector de estado apto para compresión sin pérdida	El simulador debe representar el vector de estado como arreglos contiguos de valores de punto flotante en dobles precisión (amplitudes complejas), de manera que la compresión sin pérdida se pueda aplicar directamente sobre dicha estructura sin cambios semánticos.	0.10
C2	Facilidad de integración de estructuras o arreglos personalizados	El código debe permitir reemplazar o envolver el almacenamiento del vector de amplitudes (p. ej. insertar un contenedor comprimido) con impacto controlado en el resto del simulador. Este criterio prioriza código modular y rutas críticas identificables.	0.25
C3	Soporte y afinidad con optimización de memoria	Se valora que el simulador esté concebido para experimentar con estrategias de memoria (p. ej. compresión, poda/pruning, particionamiento o variantes del almacenamiento), reduciendo esfuerzo para insertar y validar la estrategia propuesta.	0.20
C4	Complejidad de construcción y dependencias	Un proceso de compilación simple y pocas dependencias facilitan reproducibilidad y portabilidad de experimentos. Se penalizan toolchains complejas cuando incrementan el esfuerzo de integración e instrumentación.	0.10
C5	Capacidad de instrumentación (tiempo y memoria)	El simulador debe permitir medir de forma fina (por secciones) el consumo de memoria y el tiempo de ejecución, idealmente en puntos cercanos al acceso al vector de estado.	0.15
C6	Cobertura funcional mínima para casos de prueba	Debe ser posible ejecutar circuitos estándar de evaluación (p. ej. QFT y/o Grover) con un conjunto mínimo de compuertas, para poder comparar desempeño con y sin compresión.	0.10
C7	Paralelismo relevante (CPU, OpenMP, MPI)	Se considera valioso poder evaluar la interacción compresión-paralelismo. Se prioriza soporte a OpenMP/MPI cuando el objetivo incluye escalamiento o comparación bajo concurrencia.	0.10

Los pesos fueron definidos para privilegiar la **integración del almacenamiento del vector de estado** (C2) y la **afinidad con experimentación en memoria** (C3), dado que son los factores que más reducen el riesgo técnico de insertar compresión sin pérdida sin distorsionar el comportamiento del simulador.

6.3 Simuladores considerados

A continuación, se resumen las herramientas evaluadas, enfatizando sus objetivos y características relevantes para este trabajo.

- **QuEST (Quantum Exact Simulation Toolkit):** Es un simulador de alto rendimiento para vectores de estado y matrices de densidad, con soporte híbrido de paralelismo que puede incluir multihilo, GPU y ejecución distribuida. Se caracteriza por estar orientado a rendimiento en múltiples arquitecturas de cómputo. (Jones y cols., 2019; QuEST-Kit contributors, s.f.)
- **Intel-QS (Intel Quantum Simulator):** Simulador de circuitos optimizado para aprovechar arquitecturas multinúcleo y multinodo; usa representación completa del estado, y está diseñado para escalar mediante paralelismo y distribución. (Intel-QS project, s.f.)
- **qsim (Google):** Librería de C++/Python para simulación de vector de estado completo, ampliamente usada en integración con Cirq. En su documentación se describe explícitamente como simulador de estado completo, donde el costo crece proporcionalmente al número de amplitudes 2^n . (quantumlib contributors, s.f.; Google Quantum AI, s.f.)
- **Quantum++:** Librería moderna en C++ (enfocada en facilidad de uso y portabilidad) para simulación de procesos cuánticos; se distribuye como librería *header-only* y ofrece ejecución multihilo, siendo útil para prototipado. (Gheorghiu, 2018)
- **TMFQSfullstate:** Prototipo de simulador implementado en C++ con enfoque explícito en estudiar estrategias de consumo de memoria (representación de estado completo, poda de estados, paralelismo compartido y distribuido, y compresión). En particular, el trabajo metodológico que acompaña este simulador reporta escenarios con OpenMP, MPI y la incorporación de compresión para evaluar huella de memoria. (Díaz y cols.,

2024; Díaz y cols., 2025; Gilberto Díaz, Jorge Saul, Daniel González Buendía, y Javier Sarmiento, 2026)

En conjunto, las alternativas cubren desde herramientas HPC maduras (QuEST, Intel-QS, qsim) hasta librerías generales (Quantum++) y un prototipo deliberadamente diseñado para experimentar con estrategias de memoria (TMFQSfullstate).

6.4 Matriz de evaluación ponderada

Se aplicó una matriz de decisión ponderada para evaluar cada alternativa según los criterios de la tabla 1. La calificación se asignó en una escala discreta de 1 a 5, donde 5 indica cumplimiento excelente del criterio y 1 un cumplimiento deficiente para los objetivos de integración e instrumentación de compresión sin pérdida. Cada puntaje se definió a partir de la documentación técnica disponible, de las publicaciones asociadas a cada simulador y de la estructura de integración descrita para el vector de estado, de modo que la matriz funcione como un mecanismo de decisión trazable y no como una preferencia ad hoc.

Tabla 2: Evaluación de alternativas (escala 1–5) y total ponderado

Criterio	Peso	TMFQS	QuEST	Intel-QS	qsim	Quantum++
C1	0.10	5	4	4	4	4
C2	0.25	5	3	2	2	3
C3	0.20	5	3	3	2	2
C4	0.10	4	3	2	2	5
C5	0.15	5	3	2	2	3
C6	0.10	3	5	5	5	4
C7	0.10	4	5	5	4	3
Total	1.00	4.60	3.50	3.00	2.70	3.10

Los resultados muestran que, aunque QuEST, Intel-QS y qsim ofrecen excelente cobertura funcional y paralelismo, la decisión de este trabajo prioriza la capacidad de intervención

experimental sobre el almacenamiento del vector de estado y la instrumentación cercana a rutas críticas. Bajo ese objetivo, **TMFQSfullstate** obtiene el mayor puntaje global, principalmente por C2, C3 y C5.

6.5 Selección del simulador

Partiendo de la matriz de la tabla 2, se selecciona **TMFQSfullstate** como simulador base para este trabajo.

La decisión se justifica por cuatro razones principales. Esta selección no pretende afirmar que **TMFQSfullstate** sea el simulador más robusto o más eficiente en términos generales dentro del ecosistema de simulación cuántica; su propósito es escoger la plataforma con mayor **capacidad de intervención experimental** sobre el almacenamiento del vector de estado, instrumentar métricas finas y aislar el efecto de la compresión sobre memoria y tiempo.

- **Diseño orientado a experimentación de memoria (C3):** **TMFQSfullstate** se presenta como un prototipo cuyo foco explícito es evaluar consumo de memoria bajo distintas estrategias (estado completo, OpenMP, MPI y escenarios con compresión), lo cual se alinea directamente con el objetivo de insertar compresión sin pérdida sobre el vector de estado. (Díaz y cols., 2024; Díaz y cols., 2025)
- **Alta modificabilidad del núcleo (C2) e instrumentación (C5):** al ser un prototipo compacto y de propósito investigativo, reduce el esfuerzo necesario para ubicar puntos de inserción (capa de almacenamiento del vector de amplitudes y rutas críticas) y para instrumentar mediciones finas de tiempo/memoria, tal como se exige en la fase de diseño metodológico.
- **Compatibilidad con arreglos de doble precisión (C1):** el material metodológico asociado al prototipo describe implementaciones basadas en arreglos en doble precisión para compuertas y estructuras numéricas, lo que facilita plantear compresión sin pér-

dida sobre datos de punto flotante sin introducir conversiones o cuantizaciones. (Díaz y cols., 2024; Díaz y cols., 2025)

- **Escenarios comparables de paralelismo (C7):** la disponibilidad de variantes con OpenMP/MPI permite evaluar la interacción entre compresión y paralelismo, y contrastar contra simuladores HPC sin que el paralelismo sea un factor ausente en el entorno experimental. (Díaz y cols., 2024)

En consecuencia, TMFQSfullstate ofrece el mejor balance entre *control experimental* y *esfuerzo de integración*, minimizando el riesgo técnico para implementar y evaluar compresión sin pérdida en el vector de estado, manteniendo al mismo tiempo una base suficientemente realista para comparar memoria y rendimiento dentro de los criterios definidos por este trabajo.

7 Selección de la biblioteca de compresión sin pérdida

7.1 Necesidad de seleccionar una biblioteca de compresión

Un simulador tipo Schrödinger puede particionar el vector de estado en bloques independientes, mantener una caché de bloques descomprimidos y aplicar una política de descompresión bajo demanda para evitar reconstruir el estado completo en cada operación. Sin embargo, para materializar esa arquitectura en un simulador real es necesario seleccionar una biblioteca de compresión concreta que se adapte bien al acceso parcial por bloques y al patrón repetido de lectura, modificación y escritura de amplitudes.

Por esta razón, la decisión relevante es qué biblioteca de compresión sin pérdida facilita una integración eficiente en una ruta crítica de simulación cuántica. En este contexto, interesan especialmente las soluciones que trabajen con bloques independientes, permitan incorporar filtros previos sobre datos de punto flotante y minimicen la latencia de descompresión cuando solo una fracción del vector de estado debe pasar temporalmente a representación densa.

La distinción con respecto al capítulo de estado del arte es deliberada. Allí se revisaron algoritmos, transformaciones y familias de compresión sin pérdida para datos de punto flotante; en esta sección se comparan *realizaciones software concretas* que puedan integrarse en un simulador en C/C++. Por ello aparecen bibliotecas como Blosc2, Zstd, LZ4 y zlib: no todas representan un algoritmo nuevo, pero sí distintos vehículos de implementación para materializar compresión por bloques, filtros reversibles y códecs de baja latencia dentro de la arquitectura del simulador.

Espacio reservado para una figura comparando el flujo general de almacenamiento denso frente a almacenamiento por bloques comprimidos.

Figura 2: Contraste conceptual entre almacenamiento denso y almacenamiento por bloques comprimidos

7.2 Criterios de selección

La selección de bibliotecas de compresión sin pérdida se realizó mediante los criterios de la Tabla 3. Estos criterios recogen propiedades relevantes para la compresión de vectores de estado representados como arreglos numéricos de gran tamaño.

Como condiciones mínimas de inclusión se consideraron la operación sin pérdida, la disponibilidad de interfaces utilizables desde C/C++, un licenciamiento compatible con el uso académico y la posibilidad de comprimir datos residentes en memoria. Las bibliotecas comparadas satisfacen estas condiciones.

Tabla 3: Criterios para la selección de bibliotecas de compresión sin pérdida

	Criterio	Explicación	Peso
C1	Efectividad de compresión sobre arreglos numéricos	Se evalúa la capacidad de la biblioteca para reducir el tamaño de arreglos homogéneos de punto flotante, condición central en un simulador basado en vectores de estado.	0.25
C2	Velocidad de descompresión	La recuperación rápida de datos es importante cuando los valores comprimidos deben reutilizarse durante la simulación. Este criterio mide qué tan favorable es la biblioteca para escenarios sensibles a la latencia de lectura.	0.20
C3	Velocidad de compresión	Considera el costo temporal de comprimir los datos, ya que una estrategia útil para el ahorro de memoria no debe introducir una penalización excesiva en el tiempo total de simulación.	0.15
C4	Facilidad de integración e instrumentación en C/C++	La biblioteca debe ofrecer una API estable, reutilizable desde el simulador y con suficiente control sobre parámetros operativos, buffers auxiliares y niveles de configuración, reduciendo el riesgo de integración.	0.15
C5	Madurez y adopción	Se valora la estabilidad del ecosistema de la biblioteca, la disponibilidad de documentación, la adopción en entornos reales y la probabilidad de mantenimiento a mediano plazo.	0.10
C6	Adecuación a datos científicos en punto flotante	Para arreglos numéricos homogéneos, se valora la existencia o integración razonable de mecanismos que mejoren el aprovechamiento de regularidades binarias sin comprometer el carácter sin pérdida de la compresión.	0.15

Los pesos priorizan propiedades centrales del problema: ahorro efectivo de memoria sobre datos numéricos, bajo costo de acceso, integración práctica y adecuación al contexto científico.

7.3 Bibliotecas de compresión sin pérdida consideradas

Las alternativas consideradas corresponden a bibliotecas o familias de bibliotecas utilizables desde C/C++ y con potencial de integrarse en una arquitectura de almacenamiento comprimido por bloques.

- **Blosc2:** Biblioteca orientada al manejo de bloques comprimidos en memoria, con soporte para filtros como *shuffle* y *bitshuffle*, y posibilidad de usar distintos *codecs* en cada bloque. Estas características la ubican dentro del grupo de herramientas diseñadas para datos numéricos y estructuras particionadas.
- **Zstandard (Zstd):** Biblioteca generalista ampliamente utilizada en compresión sin pérdida. Se caracteriza por ofrecer distintas configuraciones de velocidad y ratio, y puede integrarse como compresor dentro de flujos de trabajo más amplios definidos por la aplicación.
- **LZ4:** Biblioteca orientada a compresión y descompresión de baja latencia. Su uso es frecuente en contextos donde la rapidez del acceso tiene un peso importante, y al igual que otras bibliotecas generalistas puede emplearse como componente dentro de esquemas de almacenamiento definidos externamente.
- **zlib/Deflate:** Solución madura y ampliamente disponible, con amplia adopción en múltiples entornos de software. Representa una referencia clásica dentro de la compresión sin pérdida y permite contrastar alternativas más recientes con una base tecnológica bien establecida.

Aunque otras alternativas también podrían adaptarse al problema, estas cuatro cubren el espectro relevante para este trabajo.

7.4 Matriz de evaluación ponderada

La evaluación se presenta mediante una matriz ponderada basada en los criterios de la Tabla 3. La escala de 1 a 5 expresa el grado de adecuación de cada biblioteca frente a compresión sin pérdida de arreglos numéricos, costo temporal de acceso e integración práctica en software científico escrito en C/C++. Las puntuaciones se asignaron a partir de la documentación oficial de cada biblioteca, de sus características de integración y del

comportamiento esperado sobre arreglos numéricos homogéneos, con el fin de dejar explícito el criterio usado en la selección.

Tabla 4: Evaluación de bibliotecas de compresión sin pérdida (escala 1–5) y total ponderado

Criterio	Peso	Blosc2	Zstd	LZ4	zlib
C1 (Efectividad en datos numéricos)	0.25	4	4	2	3
C2 (Descompresión)	0.20	5	4	5	2
C3 (Compresión)	0.15	4	4	5	2
C4 (Integración C/C++)	0.15	5	5	5	5
C5 (Madurez y adopción)	0.10	4	5	5	5
C6 (Optimización para datos numéricos)	0.15	5	2	1	1
Total	1.00	4.50	3.95	3.65	2.85

Los resultados de la matriz muestran perfiles diferenciados entre las bibliotecas evaluadas. Zstd y LZ4 presentan un buen comportamiento general como compresores de alta velocidad, zlib conserva interés como referencia madura y ampliamente adoptada, y Blosc2 obtiene la mayor puntuación total por su buen balance entre costo de acceso, integración y adecuación a arreglos numéricos homogéneos. Esta ventaja es coherente con el panorama del estado del arte: Blosc2 no sustituye por sí mismo a los algoritmos analizados allí, pero sí ofrece una realización práctica que combina compresión por bloques, filtros reversibles tipo *shuffle* y selección de códec en una interfaz directamente integrable en el simulador.

Espacio reservado para una figura o tabla complementaria que compare ratio, latencia y costo de integración de las bibliotecas evaluadas.

Figura 3: Espacio reservado para visualización comparativa de bibliotecas de compresión

7.5 Selección de la biblioteca de compresión

Con base en la Tabla 4, se selecciona **Blosc2** como biblioteca principal para la implementación del mecanismo de compresión sin pérdida.

Zstd y LZ4 presentan ventajas claras como compresores generalistas de alta velocidad, mientras que zlib ofrece una base madura y ampliamente difundida. Dentro del conjunto de criterios definidos para este trabajo, Blosc2 obtiene el mejor balance entre descompresión rápida, integración en C/C++ y adecuación al tratamiento de datos numéricos.

En consecuencia, la implementación concreta desarrollada en este trabajo adopta Blosc2 como realización de una estrategia general de compresión en memoria para vectores de estado, basada en particionamiento, acceso localizado y actualización selectiva de los datos comprimidos.

8 Patrones de compresibilidad en vectores de estado de algoritmos cuánticos

8.1 Relación entre estructura del estado y compresibilidad

La simulación cuántica de estado completo representa el sistema mediante un vector de amplitudes complejas cuyo tamaño crece como 2^n (siendo n el número de qubits). En este contexto, la *compresibilidad* del vector depende fuertemente de los patrones numéricos que generan los algoritmos: simetrías, valores repetidos, regiones con ceros exactos o distribuciones altamente estructuradas tienden a favorecer la compresión; por el contrario, estados densos con amplitudes variadas y poco redundantes tienden a reducir la efectividad de compresores sin pérdida.

En las subsecciones siguientes se describen patrones típicos de varios algoritmos conocidos y su impacto esperado en compresión de vectores de estado. Esta discusión cumple una función interpretativa y de selección de casos de prueba: no constituye por sí misma la validación experimental del trabajo. La contrastación empírica posterior se concentra en Grover y QFT, que representan dos regímenes estructurales suficientemente distintos para evaluar la estrategia propuesta.

8.2 Búsqueda de Grover: uniformidad y baja entropía

Grover inicia preparando una superposición uniforme sobre $N = 2^n$ estados base, usualmente aplicando compuertas de Hadamard a cada qubit. En esa preparación ideal, todas las amplitudes tienen el mismo valor (magnitud constante), por lo que el vector de estado queda compuesto por un único valor repetido N veces, un caso altamente favorable para compresión sin pérdida (Grover, 1996; Szabłowski, 2021).

Tras cada iteración (oráculo + difusión), la estructura permanece altamente regular: cuando existe un único elemento marcado, el vector puede describirse con dos categorías principales de amplitud (una asociada al estado marcado y otra común al resto de estados no marcados). Esta simetría reduce el número de valores distintos en el vector, lo cual incrementa la redundancia explotable por compresión.

La misma simetría admite una reducción más fuerte cuando el objetivo es simular únicamente la evolución ideal de Grover con un conjunto fijo de estados marcados. Si W denota el conjunto de estados objetivo y $M = |W|$, el estado puede expresarse en el subespacio generado por

$$|w\rangle = \frac{1}{\sqrt{M}} \sum_{x \in W} |x\rangle, \quad |r\rangle = \frac{1}{\sqrt{N-M}} \sum_{x \notin W} |x\rangle,$$

de modo que en cada iteración basta actualizar dos parámetros globales en lugar de las N amplitudes individuales. Esta observación no sustituye una estrategia general de almacenamiento, pero sí muestra que la estructura algorítmica de Grover permite optimizaciones adicionales cuando se preserva la igualdad de amplitud dentro de las clases marcada y no marcada. La derivación correspondiente se presenta en el Apéndice A.

De manera empírica, se ha reportado que la compresión permite escalar simulaciones de Grover significativamente más allá de lo que permitiría almacenar el vector sin comprimir. Por ejemplo, Wu y cols. muestran una reducción drástica del requerimiento de memoria para una simulación de Grover de 61 qubits al combinar técnicas de compresión con ejecución en supercomputación (Wu y cols., 2019).

8.3 Transformada cuántica de Fourier: variación de fase y baja redundancia

La Transformada Cuántica de Fourier (QFT) aplicada a un estado base $|k\rangle$ produce una superposición con magnitudes uniformes pero fases dependientes del índice. En su forma ideal, la amplitud de cada componente $|x\rangle$ puede expresarse como:

$$a_x = \frac{1}{\sqrt{N}} e^{\frac{2\pi i k x}{N}},$$

lo cual implica que, aunque la magnitud sea constante, las partes real e imaginaria varían de forma oscilatoria con x .

Desde la perspectiva de almacenamiento, este comportamiento tiende a generar arreglos *densos* donde la mayoría de entradas son no nulas y muchas de ellas difieren entre sí. En consecuencia, la redundancia directa (por repetición exacta) suele ser baja para compresión estrictamente sin pérdida. En un estudio de integración de compresión en simulación cuántica se observa que, a medida que avanza un circuito QFT, el ratio de compresión puede caer desde valores altos en etapas iniciales hasta valores cercanos a 1 (poca o nula compresión efectiva) en etapas finales (Wu y cols., 2018). Este comportamiento ilustra que los estados generados por QFT pueden ser un caso exigente para compresión sin pérdida, especialmente cuando el circuito densifica el vector de estado.

8.4 Algoritmo de Shor: periodicidad y concentración espectral

Shor combina (i) una etapa de exponenciación modular que induce periodicidad y (ii) una QFT que transforma esa periodicidad en una distribución con picos relacionados con el período. En términos cualitativos, antes de aplicar QFT suelen aparecer patrones donde solo ciertos índices compatibles con una estructura periódica presentan amplitud significativa; dependiendo de la instancia, esto puede producir regiones con ceros exactos o repeticiones regulares que favorecen la compresión.

Tras QFT, la distribución tiende a concentrarse alrededor de múltiplos específicos aso-

ciados a la frecuencia/período, generando picos dominantes y muchos valores de magnitud pequeña. En compresión estrictamente sin pérdida, la ganancia depende de si existen ceros exactos o repeticiones; cuando predominan muchos valores pequeños pero distintos, la redundancia disminuye. Estos escenarios se alinean con el fenómeno observado en circuitos que incrementan progresivamente la densidad del estado: conforme aparecen más amplitudes no nulas y variadas, el ratio de compresión disminuye (Wu y cols., 2018).

8.5 Deutsch–Jozsa y Simon: estructura y esparsidad

Varios algoritmos tempranos construyen superposiciones altamente estructuradas mediante oráculos e interferencia.

En Deutsch–Jozsa, la interferencia inducida por el oráculo y la transformación final de Hadamard concentra la amplitud sobre un subconjunto reducido de estados base cuando la función pertenece a una de las clases consideradas por el algoritmo. Este tipo de concentración implica esparsidad o repetición exacta de amplitudes, lo que favorece la compresión sin pérdida (Deutsch y Jozsa, 1992; Nielsen y Chuang, 2010).

En Simon, tras medir el registro de salida, el registro de entrada colapsa a una superposición de dos estados relacionados por una máscara secreta. Ese estado intermedio es extremadamente esparso y, por tanto, muy compresible. Posteriormente, la aplicación de compuertas de Hadamard produce una superposición uniforme sobre un subconjunto definido por una restricción lineal, donde reaparecen ceros exactos y amplitudes repetidas (Simon, 1997; Nielsen y Chuang, 2010).

8.6 Circuitos variacionales y circuitos aleatorios: entre regularidad y alta entropía

En aplicaciones NISQ, algunos circuitos variacionales producen distribuciones de amplitud sesgadas por la estructura del problema, lo cual puede inducir regularidades (por ejemplo, subconjuntos dominantes o restricciones que eliminan estados). Esto puede mejorar

la compresibilidad frente a escenarios plenamente aleatorios.

Por el contrario, circuitos aleatorios profundos tienden a generar estados altamente entrelazados y con amplitudes densas, donde la diversidad de valores reduce la redundancia. En simulación con compresión, este fenómeno se manifiesta como una caída del ratio a medida que el circuito genera más amplitudes no nulas y variadas, tal como se evidencia en evaluaciones de compresión sobre circuitos que densifican el vector de estado (por ejemplo, QFT) (Wu y cols., 2018).

8.7 Implicaciones para la compresión sin pérdida

Los ejemplos anteriores sugieren una heurística práctica: algoritmos que preservan simetrías fuertes (valores repetidos), esparsidad (ceros exactos) o distribuciones con pocas categorías de amplitud tienden a ser más favorables para compresión sin pérdida. En cambio, transformaciones que producen estados densos y con amplitudes altamente variadas limitan la efectividad de compresores sin pérdida.

Esta relación patrón-compresibilidad es un insumo útil para diseñar experimentos y seleccionar casos de prueba representativos al evaluar técnicas de reducción de memoria en simuladores cuánticos. Resultados experimentales sobre compresión en simulación de circuitos muestran precisamente que, en presencia de estados densos y sin reglas simples de distribución, el ratio puede acercarse a 1 (Wu y cols., 2018), mientras que en algoritmos con alta regularidad (como Grover) pueden observarse reducciones de memoria sustanciales (Wu y cols., 2019).

9 Diseño de la arquitectura de compresión sin pérdida para simuladores tipo Schrödinger

Una vez seleccionado el simulador base y definida la biblioteca de compresión a emplear, el siguiente paso consiste en establecer una arquitectura de simulación que permita introducir compresión sin pérdida sin alterar la semántica del simulador de estado completo. En esta

sección se presenta una propuesta de diseño en niveles de abstracción, pensada para ser reutilizable en cualquier simulador tipo Schrödinger cuyo estado cuántico se represente como un vector de amplitudes complejas en doble precisión.

Espacio reservado para un diagrama general de la arquitectura propuesta, con capas de simulación lógica, almacenamiento del vector de estado, gestión de compresión y orquestación de accesos.

Figura 4: Arquitectura abstracta para integración de compresión sin pérdida en simuladores tipo Schrödinger

9.1 Objetivos de diseño

El diseño propuesto busca satisfacer cuatro objetivos simultáneos. En primer lugar, reducir la huella de memoria del vector de estado, que crece exponencialmente con el número de qubits. En segundo lugar, preservar la exactitud numérica del simulador, evitando pérdidas de precisión en las amplitudes complejas. En tercer lugar, impedir que cada compuerta obligue a reconstruir el vector de estado completo en memoria densa. Finalmente, se requiere que la sobrecarga temporal introducida por la compresión permanezca controlada y pueda ser ajustada mediante parámetros de diseño.

Estos objetivos conducen a una separación clara entre la lógica del simulador y la estrategia de almacenamiento. La lógica cuántica continúa operando sobre amplitudes complejas, compuertas y mediciones, mientras que la representación física del vector puede variar entre una forma densa y una forma comprimida por bloques, siempre que ambas expongan el mismo comportamiento observable.

9.2 Representación por bloques del vector de estado

La decisión estructural principal consiste en particionar el vector de estado en bloques independientes de tamaño fijo. Cada bloque agrupa un número determinado de estados base consecutivos y almacena sus partes real e imaginaria como un arreglo contiguo de números en punto flotante. Esta organización convierte el problema global de compresión del estado

completo en una colección de problemas locales sobre subarreglos más pequeños.

La ventaja de esta representación es doble. Por una parte, cada bloque puede comprimirse y descomprimirse sin depender del contenido de los demás, lo que facilita actualizaciones localizadas. Por otra, el tamaño del bloque se convierte en un parámetro de diseño que permite balancear dos efectos opuestos: bloques más grandes tienden a favorecer el ratio de compresión, mientras que bloques más pequeños reducen el costo de acceso cuando solo una fracción del vector es requerida.

Espacio reservado para una figura del vector de estado particionado en bloques, indicando el mapeo entre índices de estados base y bloques comprimidos.

Figura 5: Particionamiento abstracto del vector de estado en bloques independientes

9.3 Descompresión selectiva y estrategia de acceso

En una arquitectura densa tradicional, cualquier compuerta opera directamente sobre el arreglo completo de amplitudes. En contraste, cuando el estado está comprimido por bloques, la política de acceso debe asegurar que solo se materialicen en memoria densa los bloques realmente tocados por la operación actual. De este modo, una compuerta de uno o dos qubits no desencadena una descompresión global, sino un proceso local de adquisición de bloques, modificación y posterior recompresión.

Esta idea conduce a una estrategia de descompresión selectiva. Cada acceso a una amplitud o a un conjunto de amplitudes se traduce en la identificación de los bloques involucrados. Si un bloque ya se encuentra disponible en forma descomprimida, la operación se realiza directamente sobre ese buffer. Si el bloque aún está almacenado en forma comprimida, primero se descomprime, se ejecuta la actualización necesaria y luego se conserva temporalmente en memoria de trabajo para evitar recomprimirlo de inmediato si es probable que vuelva a usarse en operaciones posteriores.

Desde el punto de vista algorítmico, esta estrategia es especialmente relevante para compuertas que inducen accesos repetidos a subconjuntos del estado, así como para secuencias

cortas de compuertas sobre qubits cercanos. En esos casos, mantener un subconjunto pequeño del vector en memoria descomprimida puede amortiguar el costo de las transiciones entre representación comprimida y representación densa.

9.4 Caché LRU de bloques descomprimidos

Para sostener la estrategia anterior se introduce una caché de bloques descomprimidos. Su función es actuar como un conjunto de trabajo temporal que almacena los bloques accedidos recientemente. Cuando una operación necesita un bloque, primero consulta la caché. Si el bloque está presente, ocurre un acierto y se evita una nueva descompresión. Si el bloque no está presente, ocurre un fallo, el bloque se recupera desde almacenamiento comprimido y se incorpora al conjunto de trabajo.

La política de reemplazo adoptada es del tipo *Least Recently Used* (LRU). Bajo esta política, cuando la caché está llena y se necesita espacio para un nuevo bloque, se expulsa el bloque cuyo último acceso sea el más antiguo. La elección de LRU es adecuada para este contexto porque muchas secuencias de compuertas presentan localidad temporal: un bloque recién usado tiene mayor probabilidad de volver a ser requerido en el corto plazo que uno accedido hace muchas operaciones.

Si el bloque expulsado fue modificado, debe recomprimirse antes de abandonar la caché. Si no fue modificado, puede descartarse sin costo de escritura. Esta distinción entre bloques limpios y sucios evita recompresiones innecesarias y reduce la sobrecarga total del sistema.

Espacio reservado para un diagrama del funcionamiento de la caché LRU, mostrando aciertos, fallos, actualización de uso reciente y expulsión de bloques sucios o limpios.

Figura 6: Esquema conceptual de la caché LRU de bloques descomprimidos

9.5 Flujo de compresión y descompresión

La compresión propuesta se entiende como un pipeline de etapas reversibles sobre cada bloque. Primero, el bloque en doble precisión se organiza como un arreglo contiguo de va-

lores. Luego puede aplicarse una transformación previa sobre la representación binaria de estos datos, por ejemplo una variante de *shuffle* o *bitshuffle*, con el objetivo de exponer regularidades entre bytes o planos de bits. Finalmente, el flujo transformado se entrega a un compresor sin pérdida de baja latencia.

El proceso inverso ocurre solo cuando el bloque es requerido por una operación. El bloque comprimido se descomprime, se invierte la transformación previa y se obtiene nuevamente el arreglo denso de amplitudes listo para ser manipulado por la lógica del simulador. La clave del diseño es que esta secuencia solo se ejecuta sobre los bloques activos y no sobre la totalidad del vector.

Para operaciones que afectan amplitudes pertenecientes a un mismo bloque, la actualización es enteramente local. Cuando la operación cruza bloques, la arquitectura requiere adquirir simultáneamente los bloques involucrados, descomprimir ambos, ejecutar la actualización y decidir luego cuáles mantener temporalmente en caché y cuáles devolver al almacenamiento comprimido. Este es el punto donde el tamaño de la caché y la granularidad de los bloques influyen de manera más visible sobre el costo temporal.

Espacio reservado para una figura del flujo “adquirir bloque – descomprimir – modificar – marcar como sucio – recomprimir” y su variante para operaciones entre bloques distintos.

Figura 7: Flujo abstracto de acceso y actualización de bloques comprimidos

9.6 Ventajas, parámetros y limitaciones del diseño

El principal beneficio de esta arquitectura es que desacopla el ahorro de memoria de la necesidad de reconstruir por completo el vector de estado en cada operación. Además, introduce parámetros claros y controlables: tamaño del bloque, número de entradas en caché, política de reemplazo y tipo de compresor sin pérdida. Esto permite adaptar la arquitectura a diferentes patrones de acceso y a distintas disponibilidades de memoria.

No obstante, la arquitectura también presenta limitaciones. Si los estados cuánticos generados por el algoritmo son poco compresibles, la reducción de memoria puede resultar mo-

desta. Del mismo modo, si las compuertas inducen accesos dispersos sobre muchos bloques distintos, la tasa de fallos de caché puede crecer y aumentar la cantidad de descompresiones. Por esta razón, el análisis experimental posterior debe estudiar el compromiso entre ratio de compresión, latencia y tamaño del conjunto de trabajo.

En conjunto, los elementos anteriores delimitan un espacio de compromiso entre ahorro de memoria, granularidad de acceso y costo temporal de compresión y descompresión. Sobre esa base, la siguiente sección presenta la materialización de estas decisiones en un mecanismo de almacenamiento comprimido apoyado en Blosc.

10 Implementación del esquema de almacenamiento comprimido basado en Blosc

Esta sección presenta el desarrollo del mecanismo de almacenamiento comprimido basado en Blosc incorporado en TMFQSfullstate. La implementación se apoya en una interfaz común de almacenamiento del vector de estado, sobre la cual coexisten una realización no comprimida y realizaciones comprimidas. La descripción se organiza alrededor de la disposición en bloques del vector de estado, la caché LRU de bloques descomprimidos y la estrategia de acceso local utilizada durante la simulación.

Espacio reservado para un diagrama de módulos del desarrollo realizado, mostrando la relación entre el registro cuántico, el esquema basado en Blosc, la disposición en bloques, la caché y el flujo de compresión/descompresión.

Figura 8: Arquitectura del desarrollo implementado para el esquema basado en Blosc

10.1 Alcance de la implementación

La integración se realizó sobre TMFQSfullstate, manteniendo inalterada la interfaz lógica del simulador y concentrando los cambios en la capa de almacenamiento del vector de estado. Esta decisión permite que las operaciones de más alto nivel sigan trabajando con amplitudes complejas, compuertas y mediciones, mientras que la representación física del estado puede

cambiar de densa a comprimida sin afectar la semántica observable del simulador.

En ese contexto, el esquema desarrollado se apoya en tres decisiones concretas. La primera es representar el estado mediante bloques de amplitudes. La segunda es emplear Blosc2 como biblioteca de compresión sin pérdida para cada bloque. La tercera es introducir una caché LRU de bloques descomprimidos para amortiguar accesos repetidos durante la aplicación de compuertas. En la implementación actual, esta organización convive con una representación no comprimida de referencia y con una variante alterna basada en ZFP, lo que permite contrastar distintas realizaciones sin modificar el nivel algorítmico del simulador.

10.2 Realización del esquema de almacenamiento comprimido con Blosc

La implementación realizada adopta Blosc2 como realización concreta de la estrategia abstracta descrita antes. El estado cuántico completo se divide en bloques de tamaño configurable, y cada bloque se almacena de forma comprimida. Esta organización permite que la descompresión ocurra sobre una unidad local y no sobre el vector completo.

Una ventaja importante de esta elección es que Blosc2 proporciona soporte directo para compresión rápida por bloques, así como filtros previos que mejoran la compresibilidad de arreglos numéricos. En el contexto del simulador, esto se traduce en una mejor adecuación al patrón “descomprimir solo lo necesario, modificar localmente y recomprimir cuando corresponda”.

Además, la implementación expone parámetros de configuración relevantes para el experimento: número de estados por bloque, nivel de compresión, número de hilos y cantidad de entradas de caché. Estos parámetros permiten ajustar el compromiso entre uso de memoria y costo temporal, y por ello deben ser reportados explícitamente cuando se presenten resultados experimentales. En la versión actual del simulador, el esquema comprimido comparte la misma interfaz lógica que la representación no comprimida de referencia, de modo que operaciones como inicialización, lectura de amplitudes, exportación del estado completo y aplicación de compuertas se mantienen invariantes desde el punto de vista del usuario y del

algoritmo.

10.3 Disposición en bloques y mapeo del vector de estado

La primera parte del desarrollo consistió en introducir una disposición explícita del vector de estado en bloques contiguos. Cada bloque agrupa una cantidad fija de estados base y almacena sus componentes real e imaginaria en forma intercalada. Con ello, cualquier estado base puede mapearse a un índice de bloque y a un desplazamiento local dentro de ese bloque.

Esta capa de mapeo es necesaria para que el esquema identifique rápidamente qué bloque debe adquirirse ante una operación de lectura, escritura o aplicación de compuerta. También permite tratar el último bloque de manera especial cuando el número total de estados no llena exactamente una unidad completa de almacenamiento. Esta organización es la base que permite compartir una misma lógica de actualización para representaciones comprimidas y no comprimidas, variando únicamente la materialización física del bloque activo.

Espacio reservado para una figura que muestre el mapeo entre índices globales del vector de estado, bloques físicos y desplazamientos locales dentro de cada bloque.

Figura 9: Mapeo entre estados base y bloques físicos en el esquema comprimido

10.4 Caché LRU de bloques descomprimidos

El segundo componente clave del desarrollo fue la incorporación de una caché de bloques descomprimidos. Su objetivo es mantener en memoria de trabajo los bloques usados recientemente, de modo que múltiples operaciones sucesivas puedan reutilizarlos sin repetir el proceso de descompresión.

La política de reemplazo adoptada es LRU. Cada acceso actualiza la marca de uso reciente del bloque, y cuando la caché se llena se selecciona para expulsión el bloque con el acceso más antiguo. Si el bloque expulsado fue modificado, se marca como sucio y se recomprime antes de salir de la caché; si no hubo modificaciones, puede descartarse sin costo adicional de escritura.

Desde el punto de vista de la simulación, esta decisión es importante porque muchas secuencias de compuertas reutilizan un subconjunto reducido del estado. En tales casos, una caché pequeña pero bien administrada puede reducir de forma significativa la frecuencia de descompresiones y recompresiones. En la realización actual, la caché actúa además como mecanismo de *write-back*: los bloques modificados permanecen como sucios hasta el vaciado, evitando escrituras comprimidas prematuras durante una secuencia corta de operaciones.

10.5 Estrategia de acceso y actualización local

La implementación concreta sigue el siguiente esquema operativo. Cuando una operación necesita una amplitud o un par de amplitudes, primero determina los bloques implicados. Después consulta la caché. Si el bloque ya está disponible, reutiliza el buffer descomprimido. Si no está disponible, lo recupera desde almacenamiento comprimido, lo inserta en la caché y continúa con la operación.

Una vez modificado un bloque, este no se recomprime inmediatamente por defecto. En lugar de ello, se mantiene en la caché marcado como sucio hasta que sea necesario liberarlo o hasta que la operación actual requiera un vaciado explícito. Esta política evita recompresiones prematuras cuando varias compuertas consecutivas afectan el mismo conjunto de bloques.

Para compuertas cuyas amplitudes involucradas residen en un mismo bloque, el procedimiento es completamente local. En cambio, para operaciones que afectan amplitudes distribuidas en bloques distintos, el esquema debe adquirir ambos bloques y gestionar su permanencia en la caché. Este caso hace evidente la necesidad de mantener al menos dos bloques activos y explica por qué la caché no puede reducirse a una sola entrada.

Espacio reservado para una secuencia de pasos de acceso: localizar bloque, verificar caché, descomprimir, modificar, marcar como sucio y eventualmente recomprimir al expulsar.

Figura 10: Secuencia operativa del esquema basado en Blosc

La misma infraestructura permite exportar el vector completo de amplitudes una vez fi-

naliza la ejecución del circuito. Esta capacidad resulta esencial para la validación del trabajo, ya que permite contrastar la representación comprimida frente a la referencia no comprimida sobre exactamente la misma instancia experimental y verificar si la igualdad de amplitudes se preserva o, en el caso con pérdida, cuantificar la discrepancia introducida.

El mecanismo descrito en este capítulo establece la base operativa para medir el efecto de la compresión sobre circuitos representativos. La sección siguiente presenta la evaluación experimental realizada sobre Grover y QFT, así como la comparación cuantitativa entre la estrategia sin compresión, la estrategia sin pérdida basada en Blosc y una estrategia con pérdida controlada basada en ZFP.

10.6 Optimización específica para Grover mediante parametrización reducida

Además del esquema general basado en bloques, se incorporó una variante especializada para las familias de Grover consideradas en la evaluación. Esta variante aprovecha que, con preparación uniforme y un conjunto fijo de estados marcados, la evolución ideal conserva dos clases de amplitud: una común a los estados marcados y otra común a los estados no marcados.

En lugar de materializar el vector completo de tamaño N , la simulación mantiene únicamente las amplitudes representativas de ambas clases, denotadas u_t y v_t , junto con el valor de M , número de estados marcados. La actualización de una iteración completa de Grover se reduce entonces a la recurrencia

$$\begin{bmatrix} u_{t+1} \\ v_{t+1} \end{bmatrix} = \begin{bmatrix} \frac{N-2M}{N} & \frac{2(N-M)}{N} \\ -\frac{2M}{N} & \frac{N-2M}{N} \end{bmatrix} \begin{bmatrix} u_t \\ v_t \end{bmatrix},$$

equivalente a una actualización de dimensión constante determinada únicamente por N y M . La derivación de esta expresión se detalla en el Apéndice A.

Esta optimización no reemplaza el esquema comprimido general ni constituye una alter-

nativa general de almacenamiento para otros circuitos. Su validez depende de que el algoritmo preserve la simetría entre estados marcados y no marcados; por esa razón se utilizó como comparación complementaria en Grover, donde permite estimar el beneficio adicional que puede obtenerse al explotar directamente la estructura matemática del circuito.

11 Evaluación experimental y resultados

Esta sección presenta la evaluación experimental del mecanismo de almacenamiento comprimido integrado en el simulador. La comparación se realizó entre tres estrategias de representación del estado cuántico: almacenamiento no comprimido, compresión sin pérdida mediante Blosc y compresión con pérdida controlada mediante ZFP. El análisis se organizó sobre dos familias de circuitos con perfiles de acceso contrastantes al vector de estado: búsqueda de Grover y transformada cuántica de Fourier (QFT).

La campaña experimental cubrió 22, 23 y 24 qubits. En Grover se evaluaron instancias con objetivo único y multiobjetivo; en QFT se estudiaron una superposición periódica y una superposición de alta entropía con fases aleatorias. Esta combinación permite contrastar casos con alta regularidad estructural frente a entradas donde la compresión dispone de poca redundancia explotable.

Las tablas de resultados reportan memoria residente máxima, ahorro de memoria respecto a la referencia no comprimida, tiempo total de ejecución y tiempo relativo expresado en múltiplos de la referencia. En Blosc, al tratarse de una estrategia sin pérdida, el error máximo absoluto frente a la referencia es cero. Este resultado se obtuvo comparando amplitud por amplitud el vector completo exportado por la ejecución densa y el vector completo exportado por la ejecución con Blosc sobre la misma carga de trabajo. En ZFP, la columna de error reporta el máximo error absoluto medido mediante la misma comparación completa frente a la referencia no comprimida. En la comparación complementaria con la variante optimizada de Grover, en cambio, la lectura se concentra en memoria y tiempo, ya que el propósito de esa subsección es aislar el efecto de comprimir un esquema cuya ventaja principal proviene de la reducción analítica del estado.

En el caso de Grover se añade una comparación complementaria con la variante de parametrización reducida descrita en la Sección 10.6. Su propósito es delimitar cuánto puede reducirse el costo computacional cuando la simulación explota directamente la simetría global

del algoritmo, más allá del efecto atribuible al esquema de almacenamiento.

11.1 Diseño experimental

La campaña experimental se definió a partir de cargas de trabajo explícitas para cada circuito, con el fin de separar escenarios con alta regularidad de escenarios adversos para la compresión del vector de estado. En todos los casos se evaluaron 22, 23 y 24 qubits. Cada comparación entre estrategias se realizó sobre la misma instancia del circuito, con los mismos parámetros de entrada y bajo la misma política de recolección de métricas.

En Grover se analizaron dos casos. El primero correspondió a un oráculo con objetivo único y se ejecutó con tres posiciones representativas del estado marcado: $N/8$, $N/2$ y $7N/8$, donde $N = 2^n$ es el tamaño del espacio de búsqueda. El segundo correspondió a un oráculo multiobjetivo con tres estados marcados ubicados en esas mismas regiones. En QFT se consideraron dos familias de entrada: una superposición periódica y una superposición de alta entropía con amplitudes de magnitud uniforme y fases aleatorias, $a_y = e^{i\phi_y}/\sqrt{N}$ con $\phi_y \sim U[0, 2\pi)$ y semilla fija.

Las ejecuciones se realizaron en la estación de trabajo utilizada para el desarrollo del simulador: Rocky Linux 10.1, procesador AMD Ryzen 7 5700X de 8 núcleos y 16 hilos, y 32 GiB de memoria RAM. La campaña se instrumentó con los scripts de perfilado del repositorio del simulador, que ejecutan los binarios experimentales y recolectan tiempo transcurrido y memoria residente máxima mediante `psrecord`. En Grover, cada fila resume tres instancias por tamaño y estrategia; en QFT, cada fila corresponde a una instancia por familia de entrada, número de qubits y estrategia.

Tabla 5: Cargas de trabajo utilizadas en la evaluación

Circuito	Caso	Qubits	Definición de la entrada
Grover	Objetivo único	22, 23 y 24	Tres instancias por tamaño con estado marcado en $N/8$, $N/2$ y $7N/8$.
Grover	Multiobjetivo	22, 23 y 24	Tres estados marcados por tamaño, ubicados en $N/8$, $N/2$ y $7N/8$.
QFT	Superposición periódica	22, 23 y 24	Estado de entrada con patrón regular y repetitivo.
QFT	Superposición de alta entropía	22, 23 y 24	Estado de entrada con magnitud uniforme y fases aleatorias.

Las estrategias comprimidas se evaluaron con configuraciones efectivas fijas, resumidas en la Tabla 6. Estas configuraciones corresponden a los parámetros finalmente resueltos por el simulador para las cargas de trabajo Grover y QFT, bien sea por valores por defecto o por la sintonía automática guiada por el tipo de acceso al estado. La estrategia de referencia corresponde al almacenamiento no comprimido del simulador y sirve como base para calcular las razones de memoria y tiempo reportadas en las tablas de resultados.

Tabla 6: Configuraciones de compresión utilizadas en toda la campaña experimental

Estrategia	Configuración
Blosc	<code>chunkStates = 32768</code> , <code>gateCacheSlots = 8</code> , <code>clevel = 1</code> , <code>nthreads = 4</code> , <code>compcode = 1</code> , <code>useShuffle = true</code> .
ZFP	Modo <code>FixedPrecision</code> , <code>precision = 40</code> , <code>chunkStates = 32768</code> , <code>gateCacheSlots = 8</code> , <code>nthreads = 4</code> .

Las métricas principales fueron el tiempo total de ejecución y la memoria residente máxima. Ambas se obtuvieron a partir de las trazas `psrecord`: el tiempo corresponde al último valor registrado en la columna *Elapsed time* y la memoria máxima al máximo de la columna *Real (MB)*. A partir de estas mediciones se definieron dos indicadores de lectura directa:

$$\text{Ahorro de memoria (\%)} = 100 \left(1 - \frac{M_{\text{comp}}}{M_{\text{ref}}} \right), \quad \text{Tiempo relativo (x)} = \frac{T_{\text{comp}}}{T_{\text{ref}}}.$$

En estas expresiones, M_{ref} y T_{ref} corresponden a la ejecución con almacenamiento no comprimido del mismo circuito y del mismo número de qubits. Un ahorro de memoria negativo indica que la estrategia evaluada consumió más memoria que la referencia. Para Blosc, la exactitud se verificó exportando el vector completo de amplitudes de la ejecución comprimida y comparándolo amplitud por amplitud contra la referencia no comprimida de la misma instancia experimental; en todos los casos reportados esa comparación produjo igualdad exacta y, por consiguiente, error máximo absoluto nulo. Para ZFP se deja el campo de error explícito en las tablas con el fin de consignar el valor medido correspondiente mediante la misma comparación completa del estado. En la comparación con la variante optimizada de Grover no se reporta error, porque allí el interés ya no es contrastar exactitud entre estrategias de compresión, sino cuantificar cómo cambia la relación entre memoria y tiempo cuando la simulación explota directamente la estructura algebraica del algoritmo.

11.2 Resultados para Grover

Grover fue el circuito más favorable para la compresión del estado. En los seis escenarios evaluados, Blosc produjo los mayores ahorros de memoria y lo hizo con un patrón muy estable a medida que aumentó el número de qubits. ZFP introdujo errores máximos absolutos del orden de 10^{-14} , pero aun admitiendo esa pérdida controlada redujo menos memoria y presentó una penalización temporal mucho mayor. En consecuencia, la principal conclusión de esta subsección es que, dentro de la campaña experimental realizada, la alternativa más coherente para Grover es Blosc cuando la restricción dominante es la memoria disponible.

11.2.1 Objetivo único

La Tabla 7 resume los promedios de las tres posiciones evaluadas para el estado marcado: $N/8$, $N/2$ y $7N/8$. Las variaciones entre estas tres ubicaciones fueron pequeñas en cada tamaño, por lo que el promedio preserva adecuadamente la tendencia general del caso de objetivo único.

Tabla 7: Grover con objetivo único: promedios sobre las posiciones $N/8$, $N/2$ y $7N/8$

Qubits	Estrategia	Mem. pico (MB)	Ahorro mem. (%)	Tiempo (s)	Tiempo rel. (x)	Error
22	Referencia	71.6	0.0	7.66	1.00	0
22	Blosc	15.9	77.8	37.16	4.85	0
22	ZFP	45.5	36.4	289.30	37.78	$1,61 \times 10^{-14}$
23	Referencia	135.5	0.0	24.82	1.00	0
23	Blosc	16.2	88.1	104.85	4.22	0
23	ZFP	78.2	42.3	801.01	32.27	$1,11 \times 10^{-14}$
24	Referencia	263.7	0.0	65.40	1.00	0
24	Blosc	17.1	93.5	279.33	4.27	0
24	ZFP	143.1	45.7	2274.03	34.77	$8,04 \times 10^{-15}$

El patrón del caso de objetivo único es claro. Blosc ahorra entre 77.8 % y 93.5 % de memoria frente a la referencia, y el ahorro aumenta con el número de qubits. La penalización temporal permanece relativamente estable, entre $4,22\times$ y $4,85\times$. ZFP también reduce la memoria, pero solo entre 36.4 % y 45.7 %, y lo hace con un costo temporal muy superior, entre $32,27\times$ y $37,78\times$. Además, el error máximo absoluto de ZFP se mantiene entre $8,04 \times 10^{-15}$ y $1,61 \times 10^{-14}$, es decir, en un rango muy pequeño. Sin embargo, incluso admitiendo ese error, ZFP no supera a Blosc ni en memoria ni en tiempo. Esto indica que, para Grover con objetivo único, la compresión sin pérdida ofrece la relación más favorable entre capacidad de ahorro y costo adicional.

11.2.2 Multiobjetivo

La Tabla 8 reporta los resultados del caso multiobjetivo. Aunque la presencia de varios estados marcados reduce el número de iteraciones de Grover y con ello el tiempo absoluto del circuito, el patrón relativo entre estrategias permanece muy próximo al observado en el caso de objetivo único.

Tabla 8: Grover multiobjetivo con tres estados marcados

Qubits	Estrategia	Mem. pico (MB)	Ahorro mem. (%)	Tiempo (s)	Tiempo rel. (x)	Error
22	Referencia	71.6	0.0	4.46	1.00	0
22	Blosc	15.9	77.8	21.68	4.86	0
22	ZFP	45.3	36.7	170.16	38.13	$1,61 \times 10^{-14}$
23	Referencia	135.6	0.0	13.77	1.00	0
23	Blosc	16.2	88.0	60.54	4.40	0
23	ZFP	78.3	42.2	468.60	34.02	$1,11 \times 10^{-14}$
24	Referencia	263.7	0.0	40.53	1.00	0
24	Blosc	17.3	93.4	168.84	4.17	0
24	ZFP	143.3	45.7	1342.57	33.12	$8,04 \times 10^{-15}$

La lectura del caso multiobjetivo confirma la conclusión anterior. Blosc conserva un ahorro de memoria entre 77.8 % y 93.4 %, prácticamente idéntico al del objetivo único, y mantiene una penalización temporal cercana a $4,2\times-4,9\times$. ZFP vuelve a situarse en un punto intermedio de ahorro espacial, entre 36.7 % y 45.7 %, con sobrecostos temporales que superan los $33\times$ en todos los tamaños y errores máximos absolutos nuevamente del orden de 10^{-14} . La diferencia principal frente al caso de objetivo único no es cualitativa, sino cuantitativa: la referencia no comprimida requiere menos iteraciones y por ello el tiempo absoluto del circuito disminuye. Desde el punto de vista comparativo, el resultado sigue siendo el mismo: la pequeña discrepancia numérica admitida por ZFP no se traduce en una ventaja práctica frente a Blosc.

11.2.3 Comparación complementaria con la variante optimizada

La variante de parametrización reducida descrita en la Sección 10.6 se evaluó como referencia complementaria para estimar el efecto de explotar explícitamente la simetría global de Grover. En esta subsección se mantiene el mismo criterio de lectura usado en el resto del capítulo: el ahorro de memoria y el tiempo relativo se calculan frente a la referencia no comprimida de la propia variante optimizada, para el mismo caso y el mismo número de qubits. En el caso de objetivo único, cada fila corresponde al promedio de las tres posiciones

evaluadas para ese tamaño: $N/8$, $N/2$ y $7N/8$. Dado que el interés aquí es exclusivamente espacial y temporal, la tabla omite la columna de error.

Tabla 9: Variante optimizada de Grover con objetivo único

Qubits	Estrategia	Mem. pico (MB)	Ahorro mem. (%)	Tiempo (s)	Tiempo rel. (x)
22	Referencia	71.6	0.0	0.12	1.00
22	Blosc	15.8	77.9	140.33	1159.78
22	ZFP	38.9	45.7	1110.10	9174.34
23	Referencia	135.6	0.0	0.22	1.00
23	Blosc	16.1	88.1	277.28	1249.01
23	ZFP	64.0	52.8	3052.17	13748.49
24	Referencia	263.6	0.0	0.47	1.00
24	Blosc	17.0	93.6	558.54	1189.23
24	ZFP	113.2	57.1	4424.57	9420.66

Tabla 10: Variante optimizada de Grover multiobjetivo

Qubits	Estrategia	Mem. pico (MB)	Ahorro mem. (%)	Tiempo (s)	Tiempo rel. (x)
22	Referencia	71.7	0.0	0.10	1.00
22	Blosc	15.9	77.9	35.39	340.26
22	ZFP	39.0	45.6	279.05	2683.13
23	Referencia	135.6	0.0	0.26	1.00
23	Blosc	16.2	88.1	483.84	1890.01
23	ZFP	64.1	52.8	5298.98	20699.15
24	Referencia	263.7	0.0	0.47	1.00
24	Blosc	17.0	93.5	551.55	1173.51
24	ZFP	113.1	57.1	8305.50	17671.27

La tendencia cambia de forma importante respecto a la implementación principal. Dentro de la variante optimizada, Blosc conserva ahorros muy altos de memoria, entre 77.9% y 93.6%, y ZFP se sitúa entre 45.6% y 57.1%. Sin embargo, la referencia no comprimida de esta variante ya ejecuta Grover en unas pocas décimas de segundo, por lo que cualquier

compresión introduce penalizaciones temporales extremas: entre $340,26\times$ y $1890,01\times$ para Blosc, y entre $2683,13\times$ y $20699,15\times$ para ZFP en el caso multiobjetivo. En consecuencia, una vez se explota la parametrización reducida de Grover, la compresión del estado deja de ser una opción atractiva desde el punto de vista práctico, salvo en escenarios donde la memoria disponible imponga una restricción mucho más severa que el tiempo de ejecución.

11.3 Resultados para QFT

QFT mostró el comportamiento más sensible a la estructura de la entrada. Cuando el estado inicial presenta periodicidad, tanto Blosc como ZFP reducen memoria de forma apreciable. En cambio, cuando la entrada posee alta entropía y fases aleatorias, la compresión deja de ser ventajosa y puede empeorar simultáneamente la memoria y el tiempo de ejecución. Esta diferencia convierte a QFT en el caso más ilustrativo para discutir el alcance real del esquema propuesto.

11.3.1 Superposición periódica

Tabla 11: QFT con superposición periódica

Qubits	Estrategia	Mem. pico (MB)	Ahorro mem. (%)	Tiempo (s)	Tiempo rel. (x)	Error
22	Referencia	73.6	0.0	0.44	1.00	0
22	Blosc	35.7	51.4	1.52	3.48	0
22	ZFP	19.0	74.2	2.52	5.75	$3,79 \times 10^{-11}$
23	Referencia	139.6	0.0	1.21	1.00	0
23	Blosc	34.5	75.3	3.17	2.61	0
23	ZFP	25.4	81.8	5.68	4.68	$3,79 \times 10^{-11}$
24	Referencia	271.6	0.0	2.75	1.00	0
24	Blosc	92.4	66.0	6.99	2.54	0
24	ZFP	45.6	83.2	12.10	4.41	$3,79 \times 10^{-11}$

En esta familia de entrada, ambas estrategias comprimidas logran ahorros espaciales apreciables. Blosc ahorra entre 51.4% y 75.3% de memoria y mantiene un costo temporal

entre $2,54\times$ y $3,48\times$. ZFP amplía ese ahorro hasta el rango 74.2 %–83.2 %, pero eleva el tiempo a entre $4,41\times$ y $5,75\times$ la referencia e introduce un error máximo absoluto constante del orden de 10^{-11} . En consecuencia, la compresión es útil en QFT cuando la entrada conserva periodicidad, aunque la elección entre Blosc y ZFP depende de si se privilegia el mayor ahorro espacial o una combinación más conservadora de tiempo y exactitud.

11.3.2 Superposición de alta entropía

Tabla 12: QFT con superposición de alta entropía

Qubits	Estrategia	Mem. pico (MB)	Ahorro mem. (%)	Tiempo (s)	Tiempo rel. (x)	Error
22	Referencia	71.6	0.0	0.77	1.00	0
22	Blosc	85.3	-19.0	9.67	12.52	0
22	ZFP	73.1	-2.1	79.52	103.01	$3,53 \times 10^{-11}$
23	Referencia	135.7	0.0	2.00	1.00	0
23	Blosc	152.2	-12.1	20.98	10.48	0
23	ZFP	187.6	-38.2	171.47	85.65	$2,82 \times 10^{-11}$
24	Referencia	263.6	0.0	4.33	1.00	0
24	Blosc	392.8	-49.0	44.30	10.24	0
24	ZFP	409.8	-55.5	376.40	87.03	$3,79 \times 10^{-11}$

La situación se invierte por completo cuando la entrada presenta alta entropía. En este caso, ni Blosc ni ZFP ofrecen ventaja frente al almacenamiento no comprimido. El ahorro de memoria se vuelve negativo en todos los tamaños: Blosc introduce un sobre costo espacial entre 12.1 % y 49.0 %, mientras que ZFP lo lleva hasta 55.5 %. El deterioro temporal es todavía más marcado, con factores entre $10,24\times$ y $12,52\times$ para Blosc y entre $85,65\times$ y $103,01\times$ para ZFP. Los errores de ZFP permanecen en un rango pequeño, del orden de 10^{-11} , de modo que el problema no es la exactitud admitida sino la falta de compresibilidad del estado. Desde el punto de vista experimental, este es el resultado más importante de QFT: cuando el estado carece de regularidad local, la compresión del vector completo deja de ser una estrategia útil incluso si se tolera una pequeña discrepancia numérica.

11.4 Comparación global de estrategias

La lectura conjunta de Grover y QFT permite identificar dos tendencias generales dentro de la campaña experimental realizada. La primera es que la compresión resulta útil solo cuando el estado conserva suficiente regularidad estructural. La segunda es que, dentro de ese régimen favorable, Blosc ofrece la relación más equilibrada entre ahorro de memoria y sobrecarga temporal, mientras que ZFP solo resulta competitivo en escenarios específicos y a costa de introducir un error numérico pequeño pero no nulo.

Tabla 13: Síntesis comparativa frente al almacenamiento no comprimido

Carga de trabajo	Blosc: ahorro (%)	Blosc: tiempo (x)	ZFP: ahorro (%)	ZFP: tiempo (x)	ZFP: error
Grover, objetivo único	77.8 a 93.5	4.22 a 4.85	36.4 a 45.7	32.27 a 37.78	$8,04 \times 10^{-15}$ a $1,61 \times 10^{-14}$
Grover, multiobjetivo	77.8 a 93.4	4.17 a 4.86	36.7 a 45.7	33.12 a 38.13	$8,04 \times 10^{-15}$ a $1,61 \times 10^{-14}$
QFT, superposición periódica	51.4 a 75.3	2.54 a 3.48	74.2 a 83.2	4.41 a 5.75	$3,79 \times 10^{-11}$
QFT, alta entropía	-49.0 a -12.1	10.24 a 12.52	-55.5 a -2.1	85.65 a 103.01	$2,82 \times 10^{-11}$ a $3,79 \times 10^{-11}$

La evidencia experimental muestra que Grover constituye un escenario ampliamente favorable para la compresión sin pérdida. En este circuito, Blosc ahorra más de 77 % de memoria en todos los tamaños y supera 93 % en 24 qubits, con un costo temporal cercano a 4×. ZFP introduce errores máximos absolutos del orden de 10^{-14} , pero aun con esa tolerancia reduce menos memoria y exige sobrecostos temporales superiores a 30×, por lo que su ventaja práctica resulta claramente inferior dentro de esta campaña.

QFT obliga a matizar esa lectura. Cuando la entrada es periódica, ambos compresores siguen siendo útiles: Blosc aporta una mejor relación entre ahorro y tiempo, mientras que

ZFP logra el mayor ahorro espacial a costa de errores del orden de 10^{-11} y de una penalización temporal mayor. Sin embargo, cuando la entrada presenta alta entropía, la compresión fracasa como estrategia general: el ahorro de memoria se vuelve negativo y el tiempo de ejecución crece de forma muy pronunciada incluso en ZFP, pese a que el error admitido sigue siendo pequeño. En este sentido, la evaluación delimita con claridad que la conveniencia de comprimir el vector de estado no depende solo del circuito, sino de la estructura concreta del estado que debe representarse.

12 Conclusiones y trabajo futuro

12.1 Conclusiones

Este trabajo abordó el problema del crecimiento exponencial de memoria en simuladores cuánticos tipo Schrödinger mediante el diseño e integración de una estrategia de almacenamiento comprimido sin pérdida de precisión. La estrategia se materializó en una arquitectura basada en particionamiento por bloques, descompresión selectiva, caché LRU de bloques activos e integración de Blosc2 en la capa de almacenamiento de TMFQSfullstate. Con ello fue posible modificar la representación física del vector de estado sin alterar la semántica observable del simulador ni la exactitud numérica de las amplitudes.

Los resultados experimentales muestran que la utilidad de la compresión depende de la estructura efectiva del estado cuántico. En los casos de Grover, caracterizados por una alta regularidad, la estrategia basada en Blosc produjo ahorros de memoria entre 77.8 % y 93.5 % en el rango evaluado de 22 a 24 qubits, con una penalización temporal relativamente estable, del orden de $4\times$ respecto a la referencia no comprimida. En QFT con superposición periódica también se obtuvieron reducciones apreciables de memoria, aunque menores que en Grover y con una relación memoria–tiempo más sensible a la estructura de la entrada. En cambio, para QFT con superposición de alta entropía la compresión dejó de ser conveniente: el ahorro de memoria se volvió negativo y el tiempo de ejecución aumentó de forma marcada.

El contraste con ZFP permite delimitar con mayor precisión el aporte específico de la estrategia sin pérdida. En Grover, ZFP introdujo errores máximos absolutos del orden de 10^{-14} , pero aun así redujo menos memoria y exigió sobrecostos temporales muy superiores a los de Blosc. En QFT con entrada periódica, ZFP sí logró mayores ahorros espaciales, aunque al precio de errores del orden de 10^{-11} y de una penalización temporal más alta. En QFT con alta entropía, ni siquiera la admisión de esa pequeña discrepancia numérica bastó para volver útil la compresión: el ahorro de memoria se volvió negativo y el tiempo de ejecución aumentó de forma muy marcada. En contraste, Blosc preservó igualdad exacta

frente a la referencia no comprimida en todos los experimentos reportados y ofreció el mejor equilibrio entre ahorro de memoria y sobrecarga temporal cuando el estado conservó patrones favorables para la compresión.

Desde la perspectiva de los objetivos del trabajo, la evaluación confirmó que la compresión sin pérdida puede desplazar el límite práctico de memoria sin introducir discrepancias en el estado simulado, siempre que la carga de trabajo conserve suficiente regularidad estructural. También mostró que la compresión no constituye una solución universal: su conveniencia está condicionada por la compresibilidad del estado y por el patrón de acceso inducido por el circuito. Esta conclusión se refuerza con la variante de parametrización reducida para Grover, donde la explotación directa de la estructura matemática del algoritmo supera de forma amplia a cualquier estrategia general de compresión sobre el vector completo.

En conjunto, la evidencia obtenida muestra que la compresión sin pérdida no reemplaza a las optimizaciones algorítmicas específicas ni constituye una solución universal para cualquier circuito. Su aporte es más preciso y, por ello, más útil: ampliar el rango de simulaciones posibles en entornos donde la memoria es la restricción dominante, siempre que el estado cuántico conserve redundancia explotable y se requiera preservar exactamente las amplitudes.

12.2 Trabajo futuro

Una primera línea de continuación consiste en extender la estrategia propuesta a las variantes paralelas de TMFQSfullstate basadas en OpenMP y MPI. Esta extensión permitiría estudiar la interacción entre compresión, particionamiento del vector de estado, sincronización entre hilos o procesos y movimiento de datos entre memorias locales y distribuidas, con el fin de determinar si el ahorro observado en la versión actual se mantiene en escenarios de mayor escala.

Una segunda línea corresponde al desarrollo de un planificador de parámetros que seleccione de forma automática la configuración de compresión más adecuada para cada carga de

trabajo. En el estado actual, parámetros como el tamaño de bloque, el número de entradas de caché, el nivel de compresión, el uso de filtros previos y el número de hilos se fijan antes de la ejecución. Un mecanismo adaptativo podría ajustar estas decisiones según el circuito, el tamaño del estado, la tasa de aciertos en caché o la compresibilidad observada durante la simulación.

Una tercera dirección consiste en incorporar criterios de activación selectiva. Los resultados de QFT con alta entropía muestran que comprimir todo el vector de estado no siempre es beneficioso. Por ello, resulta pertinente estudiar estrategias híbridas en las que ciertos bloques permanezcan densos, otros se compriman y la compresión pueda desactivarse temporalmente cuando la redundancia del estado no compense el costo de compresión y descompresión.

También es relevante ampliar la campaña experimental a otras familias de algoritmos y estados de entrada. Algoritmos variacionales, circuitos aleatorios, simulaciones de sistemas físicos con distinta localidad y cargas de trabajo en las que la compresibilidad cambie a lo largo de la ejecución permitirían caracterizar con mayor precisión los regímenes en los que la estrategia propuesta resulta favorable o desfavorable.

Finalmente, este trabajo puede extenderse mediante la comparación con otras bibliotecas y políticas de almacenamiento. La evaluación de compresores sin pérdida alternativos, políticas de reemplazo distintas de LRU y entornos con GPU o memoria distribuida permitiría establecer si el equilibrio observado con Blosc responde a propiedades generales de la arquitectura propuesta o a particularidades de la realización concreta estudiada aquí.

A Derivación de la parametrización reducida de Grover

Este apéndice resume la derivación de la formulación reducida usada como punto de comparación para Grover en la evaluación experimental. El objetivo es mostrar, de forma compacta, por qué bajo las hipótesis habituales del algoritmo la dinámica ideal puede describirse con solo dos parámetros.

Sea $W \subseteq \{0, \dots, N-1\}$ el conjunto de estados marcados por el oráculo, con $M = |W|$. Cuando Grover parte de la superposición uniforme y el oráculo introduce únicamente un cambio de fase sobre los estados de W , el espacio relevante de evolución puede describirse mediante los vectores normalizados

$$|w\rangle = \frac{1}{\sqrt{M}} \sum_{x \in W} |x\rangle, \quad |r\rangle = \frac{1}{\sqrt{N-M}} \sum_{x \notin W} |x\rangle.$$

El estado inicial uniforme se descompone entonces como

$$|s\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle = \sqrt{\frac{M}{N}} |w\rangle + \sqrt{\frac{N-M}{N}} |r\rangle.$$

La iteración de Grover está dada por el producto del oráculo O y el operador de difusión $D = 2|s\rangle\langle s| - I$. El oráculo actúa como

$$O|w\rangle = -|w\rangle, \quad O|r\rangle = |r\rangle,$$

mientras que el operador de difusión refleja el estado respecto a la dirección $|s\rangle$. Como ambos operadores preservan el subespacio generado por $\{|w\rangle, |r\rangle\}$, cualquier estado de la evolución ideal puede escribirse como

$$|\psi_t\rangle = \alpha_t |w\rangle + \beta_t |r\rangle.$$

Si se desea recuperar las amplitudes a nivel de estado base, basta definir

$$u_t = \frac{\alpha_t}{\sqrt{M}}, \quad v_t = \frac{\beta_t}{\sqrt{N-M}},$$

de manera que cada estado marcado tiene amplitud u_t y cada estado no marcado tiene amplitud v_t . Tras aplicar el oráculo, las amplitudes pasan a ser $-u_t$ sobre W y v_t fuera de W . El promedio de amplitud en ese punto es

$$\mu_t = \frac{-Mu_t + (N-M)v_t}{N}.$$

El operador de difusión actúa componente a componente mediante la transformación $x \mapsto 2\mu_t - x$. Por tanto, para un estado marcado se obtiene

$$u_{t+1} = 2\mu_t - (-u_t) = 2\mu_t + u_t = \frac{N-2M}{N}u_t + \frac{2(N-M)}{N}v_t,$$

y para un estado no marcado

$$v_{t+1} = 2\mu_t - v_t = -\frac{2M}{N}u_t + \frac{N-2M}{N}v_t.$$

En forma matricial,

$$\begin{bmatrix} u_{t+1} \\ v_{t+1} \end{bmatrix} = \begin{bmatrix} \frac{N-2M}{N} & \frac{2(N-M)}{N} \\ -\frac{2M}{N} & \frac{N-2M}{N} \end{bmatrix} \begin{bmatrix} u_t \\ v_t \end{bmatrix}.$$

La evolución completa queda así reducida a una recurrencia lineal de dimensión dos. En consecuencia, si el objetivo es seguir la dinámica ideal de Grover bajo estas hipótesis, cada iteración puede resolverse actualizando solo los parámetros u_t y v_t , en lugar de aplicar el oráculo y la difusión sobre las N amplitudes del vector de estado. La reconstrucción explícita del vector completo solo es necesaria cuando se requiere materializar las amplitudes

individuales:

$$a_x^{(t)} = \begin{cases} u_t, & x \in W, \\ v_t, & x \notin W. \end{cases}$$

Esta reducción depende de que la preparación inicial, el oráculo y la dinámica posterior preserven la igualdad de amplitud dentro de las clases marcada y no marcada. Por ello, se trata de una optimización específica para Grover y no de un sustituto de un esquema general de simulación o almacenamiento.

Referencias

- Boixo, S., Isakov, S. V., Smelyanskiy, V. N., y Neven, H. (2017). Simulation of low-depth quantum circuits as complex undirected graphical models. *arXiv preprint arXiv:1712.05384*. Descargado de <https://arxiv.org/abs/1712.05384>
- Burtscher, M., y Ratanaworabhan, P. (2009). Fpc: A high-speed compressor for double-precision floating-point data. *IEEE Transactions on Computers*, 58(1), 18–31. Descargado de <https://userweb.cs.txstate.edu/~mb92/papers/tc09.pdf> doi: 10.1109/TC.2008.131
- Chen, J., Zhang, F., Huang, C., Newman, M., y Shi, Y. (2018). *Classical simulation of intermediate-size quantum circuits*. Descargado de <https://arxiv.org/abs/1805.01450>
- Deutsch, D., y Jozsa, R. (1992). Rapid solution of problems by quantum computation. *Proceedings of the Royal Society of London. Series A: Mathematical and Physical Sciences*, 439(1907), 553–558. Descargado de <https://doi.org/10.1098/rspa.1992.0167> doi: 10.1098/rspa.1992.0167
- Díaz, G., Steffenel, L., Barrios, C., y Couturier, J. (2025). Memory management strategies for software quantum simulators. *Quantum Reports*, 7(3), 41. Descargado de <https://www.mdpi.com/2624-960X/7/3/41> doi: 10.3390/quantum7030041
- Díaz, G., Steffenel, L. A., Barrios, C. J., y Couturier, J.-F. (2024). How to build a software quantum simulator. *Preprints*. Descargado de <https://doi.org/10.20944/preprints202409.1497.v1> doi: 10.20944/preprints202409.1497.v1
- Díaz Toro, G. J. (2025). *Gestión de memoria en simuladores de computación cuántica de software* (Tesis doctoral, Universidad Industrial de Santander). Descargado 2025-05-13, de <https://noesis.uis.edu.co/items/357883df-3223-45b0-9848-f7681c5b0511>
- Gheorghiu, V. (2018). Quantum++: A modern c++ quantum computing library. *PLOS ONE*, 13(12), e0208073. Descargado de <https://journals.plos.org/plosone/>

- `article?id=10.1371/journal.pone.0208073` doi: 10.1371/journal.pone.0208073
- Gilberto Díaz, Jorge Saul, Daniel González Buendía, y Javier Sarmiento. (2026). *buendia-gon/tmfqsfullstate: v1.0.0*. Zenodo software release. Descargado de <https://doi.org/10.5281/zenodo.19268085> (Source code archive of the TMFQSfullstate simulator) doi: 10.5281/zenodo.19268085
- Google Quantum AI. (s.f.). *qsimcirq python reference*. Official documentation. Descargado 2026-03-27, de <https://quantumai.google/reference/python/qsimcirq>
- Grover, L. K. (1996). A fast quantum mechanical algorithm for database search. En *Proceedings of the twenty-eighth annual acm symposium on theory of computing* (pp. 212–219). Descargado de <https://dl.acm.org/doi/10.1145/237814.237866> doi: 10.1145/237814.237866
- Häner, T., y Steiger, D. S. (2017). 0.5 petabyte simulation of a 45-qubit quantum circuit. En *Proceedings of the international conference for high performance computing, networking, storage and analysis (sc)*. doi: 10.1145/3126908.3126947
- Huffman, N., Pavlichin, D., y Weissman, T. (2024). *Lossy compression for schrödinger-style quantum simulations*. Descargado de <https://arxiv.org/abs/2401.11088>
- Intel-QS project. (s.f.). *Intel quantum simulator (intel-qs) documentation*. ReadTheDocs. Descargado 2026-03-27, de <https://intel-qs.readthedocs.io/en/docs/getting-started.html>
- Jamalidinan, S., y Cheshmi, K. (2025). *Floating-point data transformation for lossless compression*. Descargado de <https://arxiv.org/abs/2506.18062> doi: 10.48550/arXiv.2506.18062
- Jaques, S., y Häner, T. (2021). *Leveraging state sparsity for more efficient quantum simulations*. Descargado de <https://arxiv.org/abs/2105.01533>
- Jones, T., Brown, A., Bush, I., y Benjamin, S. C. (2019). QuEST and High Performance Simulation of Quantum Computers. *Sci. Rep.*, 9, 10736. doi: 10.1038/s41598-019-47174

- Lindstrom, P., y Isenburg, M. (2006). Fast and efficient compression of floating-point data. *IEEE Transactions on Visualization and Computer Graphics*, 12(5), 1245–1250. Descargado de <https://pubmed.ncbi.nlm.nih.gov/17080858/> doi: 10.1109/TVCG.2006.143
- Markov, I. L., y Shi, Y. (2008). Simulating quantum computation by contracting tensor networks. *SIAM Journal on Computing*, 38(3), 963–981. Descargado de <https://arxiv.org/abs/quant-ph/0511069> doi: 10.1137/050644756
- Masui, K., Amiri, M., Connor, L., Deng, M., Fandino, M., Hoefer, C., ... Vanderlinde, K. (2015). A compression scheme for radio data in high performance computing. *arXiv preprint arXiv:1503.00638*. Descargado de <https://arxiv.org/abs/1503.00638> (Describes the Bitshuffle filter and HDF5 plugin) doi: 10.48550/arXiv.1503.00638
- Miller, D. M., y Thornton, M. A. (2006). QMDD: A decision diagram structure for reversible and quantum circuits. En *Proceedings of the 36th international symposium on multiple-valued logic (ISMVL'06)* (p. 30). Descargado de <https://ieeexplore.ieee.org/document/1623982/> doi: 10.1109/ISMVL.2006.35
- Nielsen, M. A., y Chuang, I. L. (2010). *Quantum computation and quantum information* (10th Anniversary Edition ed.). Cambridge University Press.
- Pednault, E., Gunnels, J. A., Nannicini, G., Horesh, L., Magerlein, T., Solomonik, E., ... Wisnieff, R. (2017). *Pareto-efficient quantum circuit simulation using tensor contraction deferral*. Descargado de <https://arxiv.org/abs/1710.05867>
- Qiskit Development Team. (2025). *Qiskit aer documentation*. Official documentation. Descargado 2026-03-27, de <https://qiskit.github.io/qiskit-aer/>
- quantumlib contributors. (s.f.). *qsim: Fast c++ and python library for state-vector simulation of quantum circuits*. GitHub repository. Descargado 2026-03-27, de <https://github.com/quantumlib/qsim>
- QuEST-Kit contributors. (s.f.). *Quest: Quantum exact simulation toolkit*. GitHub repository. Descargado 2026-03-27, de <https://github.com/QuEST-Kit/QuEST>

- Simon, D. R. (1997). On the power of quantum computation. *SIAM Journal on Computing*, 26(5), 1474–1483. Descargado de <https://doi.org/10.1137/S0097539796298637> doi: 10.1137/S0097539796298637
- Szabłowski, P. J. (2021). Understanding mathematics of grover’s algorithm. *Quantum Information Processing*, 20(5), 182. Descargado de <https://doi.org/10.1007/s11128-021-03125-w> doi: 10.1007/s11128-021-03125-w
- Viamontes, G. F., Markov, I. L., y Hayes, J. P. (2003). Improving gate-level simulation of quantum circuits. *Quantum Information Processing*, 2(5), 347–380. Descargado de <https://deepblue.lib.umich.edu/handle/2027.42/45525> doi: 10.1023/B:QINP.0000022725.70000.4a
- Vidal, G. (2003). Efficient classical simulation of slightly entangled quantum computations. *Phys. Rev. Lett.*, 91(14), 147902. doi: 10.1103/PhysRevLett.91.147902
- Vinkhuijzen, L., Coopmans, T., Elkouss, D., Dunjko, V., y Laarman, A. (2023, septiembre). LIMDD: A Decision Diagram for Simulation of Quantum Computing Including Stabilizer States. *Quantum*, 7, 1108. Descargado de <https://quantum-journal.org/papers/q-2023-09-11-1108/> doi: 10.22331/q-2023-09-11-1108
- Wu, X.-C., Di, S., Cappello, F., Finkel, H., Alexeev, Y., y Chong, F. T. (2018). Amplitude-aware lossy compression for quantum circuit simulation. En *Proceedings of the 4th international workshop on data reduction for big scientific data (drbsd-4) at sc18*. IEEE. Descargado de https://sc18.supercomputing.org/proceedings/workshops/workshop_files/ws_drbsd112s1-file1.pdf
- Wu, X.-C., Di, S., Dasgupta, E. M., Cappello, F., Finkel, H., Alexeev, Y., y Chong, F. T. (2019). Full-state quantum circuit simulation by using data compression. En *Proceedings of the international conference for high performance computing, networking, storage and analysis (sc’19)*. IEEE/ACM. doi: 10.1145/3295500.3356155
- Zhang, B., Fang, B., Ye, F., Gu, Y., Tallent, N., Tan, G., y Tao, D. (2024). *Overcoming memory constraints in quantum circuit simulation with a high-fidelity compression fra-*

mework. Descargado de <https://arxiv.org/abs/2410.14088>